



GROUP PROJECT - REPORT

TOPIC : Web Application for Organizational Project & Team Management

Group: 30

Group members: Nguyễn Minh Đức – 23BI14102
Nguyễn Hoàng Dũng – 23BI14111
Huỳnh Anh Dũng – 23BI14114
Trịnh Quang Dũng – 23BI14115
Lưu Trần Minh Hiếu – 23BI14164
Đào Xuân Hiếu - 23BI14161

Supervisor: MSc. Huỳnh Vinh Nam

TABLE OF CONTENTS

1. INTRODUCTION 6

 1.1. Context and Motivation 6

 1.2. Objectives 6

 1.3. Organization of the Report 7

2. SYSTEM OVERVIEW & SCOPE 8

 2.1. Feature Description..... 8

 2.2. Expected Outcomes 9

3. THEORETICAL BACKGROUND 10

 3.1 Software Architecture Patterns 10

 3.2 Technology Stack 10

 3.3. Related Work..... 10

4. SYSTEM ANALYSIS AND DESIGN 12

 4.1. System Requirements 12

 4.2. Use Case Modeling..... 14

 4.3 Use Case and Scenario Description..... 15

5. METHODOLOGY 32

 5.1 System Architecture 32

 5.2. Database Design 33

 5.3. Backend Implementation..... 38

 5.4. Frontend Implementation 51

 5.5. API Documentation 55

6. RESULTS AND DISCUSSION 60

 6.1. Functionality Tests 60

 6.2. Performance Tests 64

 6.3. Visual Result 65

 6.4. Discussion..... 74

7. CONCLUSION AND FUTURE WORK..... 76

 7.1. Conclusion 76

 7.2. Future Work..... 76

References..... 78

List of Tables

Table 1: Role-Based Access Control Matrix	15
Table 2: Login Workflow	18
Table 3: Joining and Inviting Workflow	19
Table 4: Manage Task Details Basic Workflow	20
Table 5: Kanban Drag & Drop Workflow	21
Table 6: Real-time Chat Basic Workflow	22
Table 7: Comment & Mention Basic Workflow	23
Table 8: Schedule Meeting Basic Workflow	24
Table 9: AI Subtask Generation Workflow	25
Table 10: IP-Based Attendance Check-in Workflow	26
Table 11: Attendance Report Generation Workflow	27
Table 12: View Dashboard & Analytics Basic Workflow	28
Table 13: Project Managing Workflow	30
Table 14: Project management Workflow	31
Table 15: Manage Subscription Basic Workflow	32
Table 16: IP Whitelist Configuration Workflow	33
Table 17: Authentication API Endpoints	60
Table 18: Projects & Collaboration API Endpoints	61
Table 19: Task Management API Endpoints	61
Table 20: Notification & Alerts API Endpoints	62
Table 21: Dashboard Analysis API Endpoints	62
Table 22: Payment & Subscription API Endpoints	62
Table 23: Meeting & Scheduling API Endpoints	62
Table 24: Real-time Chat API Endpoints	63
Table 25: Organization API Endpoints	63
Table 26: Attendance API Endpoints	63
Table 27: Comments & Collaboration API Endpoints	64
Table 28: Functionality Tests	69
Table 29: System Performance & Load Test Summary	70

List of Figures

Figure 1: Auth & Onboarding	16
Figure 2: Role-Based Operations	17
Figure 3: High-Level System Component Architecture.....	34
Figure 4: System Entity Relationship Diagram (ERD)	35
Figure 5: Database Organization	36
Figure 6: Database OrganizationMember.....	36
Figure 7: Database User	36
Figure 8: Database Project.....	37
Figure 9: Database ProjectMember	37
Figure 10: Database Task	37
Figure 11: Database Attendance.....	38
Figure 12: Database Meeting.....	38
Figure 13: Database Notification.....	38
Figure 14: Message Notification	39
Figure 15: ActivityLogNotification.....	39
Figure 16: Database Comment	39
Figure 17: Database Label.....	40
Figure 18: User Login Sequence Diagram	41
Figure 19: Member Role Update Sequence Diagram.....	42
Figure 20: Project Context Switching Sequence Diagram	43
Figure 21: Dashboard Analytics & Data Aggregation Sequence Diagram.....	44
Figure 22: Project Join Request via Invite Code Sequence Diagram.....	45
Figure 23: Premium Plan Upgrade & Stripe Payment Sequence Diagram	46
Figure 24: Real-time Chat Sequence Diagram	47
Figure 25: Notification & Mention Sequence Diagram	48
Figure 26: Create & Edit Task Sequence Diagram	49
Figure 27: Kanban Task Reordering Sequence Diagram	50
Figure 28: Attendance System Sequence Diagram	51
Figure 29: AI - Magic Subtask & Daily Brief Sequence Diagram.....	52
Figure 30: Meeting Scheduling Sequence Diagram	53
Figure 31: Project Creation & Plan Validation Sequence Diagram	54
Figure 31 & 32: Dashboard layout for Admin	71
Figure 33: Calendar layout for Admin	72
Figure 34: Attendance Report.....	72
Figure 35: Config IPs to check attendance	73
Figure 36: Members layout for Admin.....	73
Figure 38: Create Project form	74
Figure 39: Project layout for Admin.....	74
Figure 40 & 41: Dashboard layout for Manager	75
Figure 42: Kanban Board layout for Manager.....	75

Figure 43: Form to make new task 76

Figure 44: Task detail layout for Manager and Member 76

Figure 45: New Meeting form 77

Figure 46: Notification layout for all User 78

Figure 47: Personal Information..... 78

Figure 48: Account Settings 79

1. INTRODUCTION

1.1. Context and Motivation

In the contemporary digital landscape, Small and Medium-sized Enterprises (SMEs) face increasing pressure to optimize operational efficiency and internal communication. Traditionally, many organizations have relied on fragmented and disparate tools to manage their workflows. Communication often occurs through consumer-grade applications such as Zalo or Messenger, while project management and task tracking are manually handled via static spreadsheets like Microsoft Excel or Google Sheets. This decentralized approach results in significant data silos, lack of real-time synchronization, and inefficient collaboration, ultimately hindering organizational growth and agility.

The shift towards digital transformation necessitates a move from these disjointed manual processes to centralized, cloud-based solutions. However, the existing market for enterprise resource planning (ERP) and project management software often targets large corporations with complex, expensive, and resource-intensive platforms. This creates a critical gap in the market: a specific need for a cost-effective, all-in-one solution tailored to the unique constraints and requirements of SMEs. Such a solution must integrate essential functions - communication (Chat), task management (Task), and intelligent automation (AI) - into a single, cohesive ecosystem. The motivation behind this project is to bridge this gap by developing a centralized Software-as-a-Service (SaaS) platform that democratizes access to advanced enterprise tools for SMEs. This proposed solution is realized through the development of "Syncora," an integrated collaborative platform specifically architected to meet these evolving enterprise demands.

1.2. Objectives

The main goal of this project is to design and build a collaborative platform that helps teams work more efficiently by reducing the problems caused by using many separate tools. Instead of relying on different applications for chat, task tracking, and reporting, this system aims to bring everything together in one place. The objectives of the project can be divided into two main groups: business objectives and technical objectives.

Business Objectives:

- **Centralize Operations:** The platform aims to combine communication and task management into a single system. By doing so, users no longer need to switch between multiple applications, which helps reduce confusion and improves daily workflow efficiency.
- **Reduce Switching Costs:** The system is designed to be easy to use and quick to adopt. A simple and intuitive interface helps teams get started faster and lowers the effort and cost usually required to learn and deploy complex enterprise software.
- **Enhance Productivity with AI:** Artificial Intelligence is used to support users in their daily work. The system can automatically break down tasks, summarize information, and provide helpful suggestions, allowing team members to focus more on important work and decision-making.

Technical Objectives:

- **Scalable Multi-tenant Architecture:** The system is built as a multi-tenant SaaS platform that can serve multiple organizations at the same time. This architecture ensures that the platform can grow as more users and organizations join, while still maintaining stable performance.
- **Data Isolation and Security:** Each organization's data is strictly separated from others. This helps protect user information, ensures privacy, and prevents unauthorized access between different organizations.
- **Low-latency Real-time Communication:** The platform includes real-time features such as chat, notifications, and live task updates. By using technologies like WebSockets, the system ensures that these interactions happen quickly and smoothly, with minimal delay.

1.3. Organization of the Report

- **Chapter 1: Introduction** – Introduces the project context, identifies challenges faced by SMEs, and defines the primary objectives.
- **Chapter 2: System Overview & Scope**– Outlines the system scope and details core functional modules, including Task Management and AI integration.
- **Chapter 3: Theoretical Background** – Reviews theoretical foundations, related works, and the technology stack supporting the development.
- **Chapter 4: System Analysis and Design**– Analyzes the system requirements, focusing on functional specifications and Use Case scenarios.
- **Chapter 5: Methodology** – Covers the complete technical construction, ranging from System Architecture and Database Design to the detailed implementation of Backend and Frontend components.
- **Chapter 6: Results and Discussion** – Presents the project outcomes through functionality and performance testing, supported by visual results and a critical discussion of the system's capabilities.
- **Chapter 7: Conclusion and Future Work** – Summarizes the main contributions and proposes directions for future enhancements.

2. SYSTEM OVERVIEW & SCOPE

2.1. Feature Description

The system provides a steady foundation for project coordination through the following core functionalities:

FEATURE 1: Authentication & Onboarding

- **SUB-FEATURE 1.1: Register:** allow users to create a new account to access the system.
- **SUB-FEATURE 1.2: Login:** provide secure access via traditional credentials or Google OAuth2.0.
- **SUB-FEATURE 1.3: Organization Plans:** manage organizational tiers, distinguishing between Premium and Free plans.
- **SUB-FEATURE 1.4: Join Request System:** Join projects via code with manager approval.

FEATURE 2: Core Task Management

- **SUB-FEATURE 2.1: Kanban Board:** organize project workflows into four distinct columns: Backlog, Todo, In Progress, and Done.
- **SUB-FEATURE 2.2: Task CRUD:** allow users (managers) to create new tasks within a specific project while ensuring strict organizational data isolation.
- **SUB-FEATURE 2.3: Task Reordering:** Drag-and-drop tasks using smart indexing.
- **SUB-FEATURE 2.4: Project Labeling:** allow tasks to be categorized using customizable, project-scoped labels.
- **SUB-FEATURE 2.5: Tiered Authorization:** restrict task modifications based on user roles and specific task assignments.

FEATURE 3: Collaboration

- **SUB-FEATURE 3.1: Real-time Chat:** enable instant communication via secure, room-based socket connections dedicated to each project.
- **SUB-FEATURE 3.2: Mention System:** allow users to tag team members in comments using the "@" prefix to trigger direct alerts.
- **SUB-FEATURE 3.3: Push Notifications:** deliver real-time updates for task assignments, mentions, and approaching deadlines.
- **SUB-FEATURE 3.4: Task Attachments:** support the management of external file links and resource attachments within specific task contexts.
- **SUB-FEATURE 3.5: Meeting Scheduling:** Schedule meetings and invite project members.

FEATURE 4: AI Productivity

- **SUB-FEATURE 4.1: Magic Subtask:** AI breaks tasks into actionable steps.
- **SUB-FEATURE 4.2: Daily Smart Summary:** AI generates personalized daily work briefings.

FEATURE 5: Monitoring & Analytics

- **SUB-FEATURE 5.1: IP Check-in:** Attendance validation against authorized office IP addresses to ensure verified and secure check-ins.
- **SUB-FEATURE 5.2: Attendance Reports:** Access, filter, and review comprehensive member check-in history, including detailed records of lateness and absences.
- **SUB-FEATURE 5.3: Dashboard Analytics:** Visualize real-time project health, task completion rates, and member activity logs through an interactive summary dashboard.

FEATURE 6: Project & User Administration

- **SUB-FEATURE 6.1: Member Management:** Review and approve join requests, search members by name or email, view detailed profiles, assign project roles, and perform soft-deletes on accounts.
- **SUB-FEATURE 6.2: Project Operations:** Initialize, configure, or delete organization projects with strict data isolation and the ability to assign specific users to projects.
- **SUB-FEATURE 6.3: 3-Level RBAC:** Enforce strict resource access control across three hierarchical tiers: Owner, Manager, and Member.

FEATURE 7: Organization Configuration

- **SUB-FEATURE 7.1: Billing Management:** Manage organizational tiers and upgrade plans (Free/Premium) to unlock system limits and advanced capabilities.
- **SUB-FEATURE 7.2: IP Configuration:** Configure and manage whitelisted IP addresses to maintain security standards for attendance verification.

2.2. Expected Outcomes

The project aims to deliver a high-quality management platform with the following expected results:

- Develop a backend application that includes all task tracking, real-time communication, and AI-powered features discussed.
- Develop a user-friendly web interface that allows team members to interact with the system, manage their work, and collaborate with others easily.
- Ensure a smooth and reliable experience for users to stay productive and work together effectively without interruptions.
- The web application should be able to run smoothly on multiple popular browsers such as Chrome, Firefox, and Safari.

3. THEORETICAL BACKGROUND

3.1 Software Architecture Patterns

Multi-tenancy is an instance of the software that supports infrastructure that serves several customers but shares within a database. Each tenant's data is isolated and invisible to other tenants.

In the context of this project, we adopted the Share Database and Shared Schema strategy, in order to optimize for scalability. Since data is stored in a shared environment, we implement logical data isolation at the application layer.

Every incoming API request is intercepted by a security middleware, which extracts the user's context from the JWT. Our system automatically injects a filter into every database query. This creates a boundary, guaranteeing that a user from Organization A cannot access Organization B.

3.2 Technology Stack

The system is built upon the MERN Stack (MongoDB, Express.js, React, Node.js).

To be more specific, Node.js serves as the backbone of our backend due to its Event-driven, Non-blocking I/O model. This handles these concurrent connections and ensures high responsiveness since there are thousands of concurrent actions occurring, traditional architectures may struggle with latency.

We utilize Socket.io for communication features. This allows the server to push updates such as chat messages or notification to the client instantly. This architecture ensures all members see the state in real-time.

For the "Magic Subtask" feature and Daily summary task, we integrate Llama 3.3, a state-of-the-art Large Language Model (LLM), accessed via the Groq Cloud API. A challenge in AI integration is ensuring the output is machine-readable. We utilize strict System Prompt Engineering to instruct Llama 3.3 to generate responses solely in valid JSON format. This allows the backend to parse the AI's suggestions and insert it into the MongoDB database as structured Task objects.

3.3. Related Work

To position our project effectively, we analyzed the strengths and limitations of three market leaders: Trello, Jira, and Asana. This analysis highlights the specific market gaps our system aims to fill.

3.3.1. Landscape of Existing Solutions

- Trello: While user-friendly, Trello lacks native hierarchical permissions (e.g., distinguishing Admins from Members) and deep reporting capabilities, limiting its utility for secure, data-driven management.
- Jira: Although the industry standard for Agile development, its high complexity and cost create a steep barrier to entry for non-technical SMEs.
- Asana: Effective for general task tracking but operates in isolation, lacking essential business integrations such as HR management or native Generative AI.

3.3.2. Gap Analysis & Strategic Improvements

Our system integrates three key features often missing or fragmented in the analyzed tools:

- **Contextual Communication:** Unlike competitors that force context-switching to external apps like Slack, our system integrates Real-time Chat directly within the project scope. This ensures collaboration remains focused and tethered to the actual work.
- **Democratized AI Access:** While major platforms often gate AI behind enterprise paywalls, we implement AI Magic Subtasks as a core utility. This allows all users to leverage GenAI for breaking down complex tasks without premium subscriptions.
- **Unified Operations (HR Integration):** To bridge the gap between task execution and workforce management, we incorporate IP-based Attendance Check-in. This feature enables managers to track both project progress and employee availability within a single dashboard, eliminating the need for separate HR software.

4. SYSTEM ANALYSIS AND DESIGN

4.1. System Requirements

4.1.1. Functional Requirements

The system functionality is defined by a strict permission model and specific business rules to ensure operational integrity and data security.

a. Permission Matrix

The following matrix defines the operational scope for each user role, ensuring appropriate access control across the system modules.

Module	Feature / Action	Org Admin (Owner)	Project Manager	Member
Billing & Subscription	Upgrade to Premium / Cancel Subscription	Yes	No	No
	View Billing History & Invoices	Yes	No	No
Organization Management	Configure Whitelist IPs	Yes	No	No
Project Management	Create New Project	Yes	No	No
	Archive / Delete Project	Yes	No	No
	Generate / Reset Invite Code	Yes	Yes	No
Team Management	Remove Member from Project	Yes (All roles)	Yes (Lower roles)	No
	Approve Join Requests	Yes	Yes	No
	Update Member Roles	Yes	Yes	No
Communication & Meeting	Create / Delete Meetings	Yes	Yes	No
	View Meetings	Yes	Yes	Yes
	Chat in Project	Yes	Yes	Yes

Task Management	Create / Assign Tasks	Yes	Yes	No
	Update Task Status (Todo/Doing/Done)	Yes	Yes	Yes
	View Task Analytics (Team Performance)	Yes	Yes	No
Dashboard	View Organization KPIs (Revenue, Growth)	Yes	No	No
	View Project-level Statistics	Yes	Yes	No
	View Personal Performance	Yes	Yes	Yes

Table 1: Role-Based Access Control Matrix

b. Business Rules

To maintain system stability and fair usage, the following operational rules are strictly enforced:

- **Plan Limits :**
 - **Free Plan:** Each organization is restricted to creating only 1 active project. To create additional projects, the owner must upgrade the subscription.
 - **Premium Plan:** Removes all restrictions, allowing the creation of unlimited projects immediately after the upgrade is confirmed.
- **Attendance Security:**
 - Employees are only allowed to "Check-in" if their device is connected to the company's verified network (Whitelisted IPs). Any check-in attempt from an outside location (e.g., home or cafe) will be blocked to prevent fraud.
- **Data Privacy (Chat System):**
 - **Project-Based Privacy:** Communication is strictly isolated within each project.
 - **Access Control:** A user can only view chat history and send messages in projects where they are an active member. Accessing data from projects they do not belong to is strictly prohibited.

4.1.2. Non-functional Requirements

- **Users Requirement:** Have a device with internet access, preferable Chrome web browser
- **Security:** Implements a secure stateless authentication mechanism using JSON Web Tokens (JWT). The system enforces a strict 3-level Role-Based Access Control (RBAC) to protect organizational data, with passwords secured via Bcrypt hashing.
- **Performance:** Targets a low latency for core interactions. Real-time features, including the Kanban board and project chat, utilize Socket.io to ensure instantaneous updates.

- **Reliability & Data Integrity:** Ensures data accuracy through atomic operations and consistent activity logging during critical workflows.
- **Scalability:** Utilizes a stateless Node.js architecture and optimized, decoupled database collections to support horizontal expansion as the user base grows.

4.2. Use Case Modeling

4.2.1. Use Case Diagram

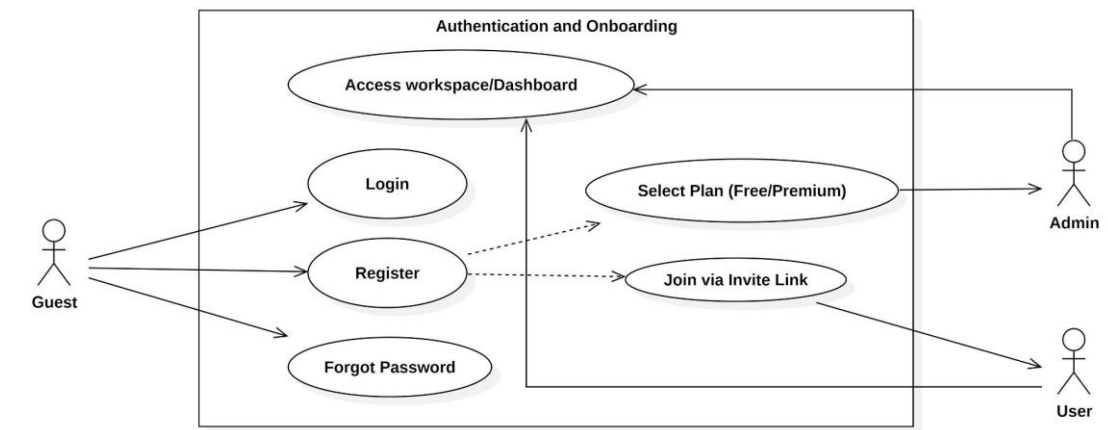


Figure 1: Auth & Onboarding

This diagram illustrates the logical flow for authentication and authorization while enforcing organizational subscription constraints.

Key Design Logic:

- **Plan-Based Validation:** The system performs a quota check to distinguish between Free and Premium organizational tiers before proceeding with creation.
- **Quota Enforcement:** For Free plans, the system strictly restricts creation if the project count exceeds one, returning a "403 Forbidden" error status.
- **Administrative Auditing:** Every successful project initialization is logged as an administrative activity in the database for tracking and security.

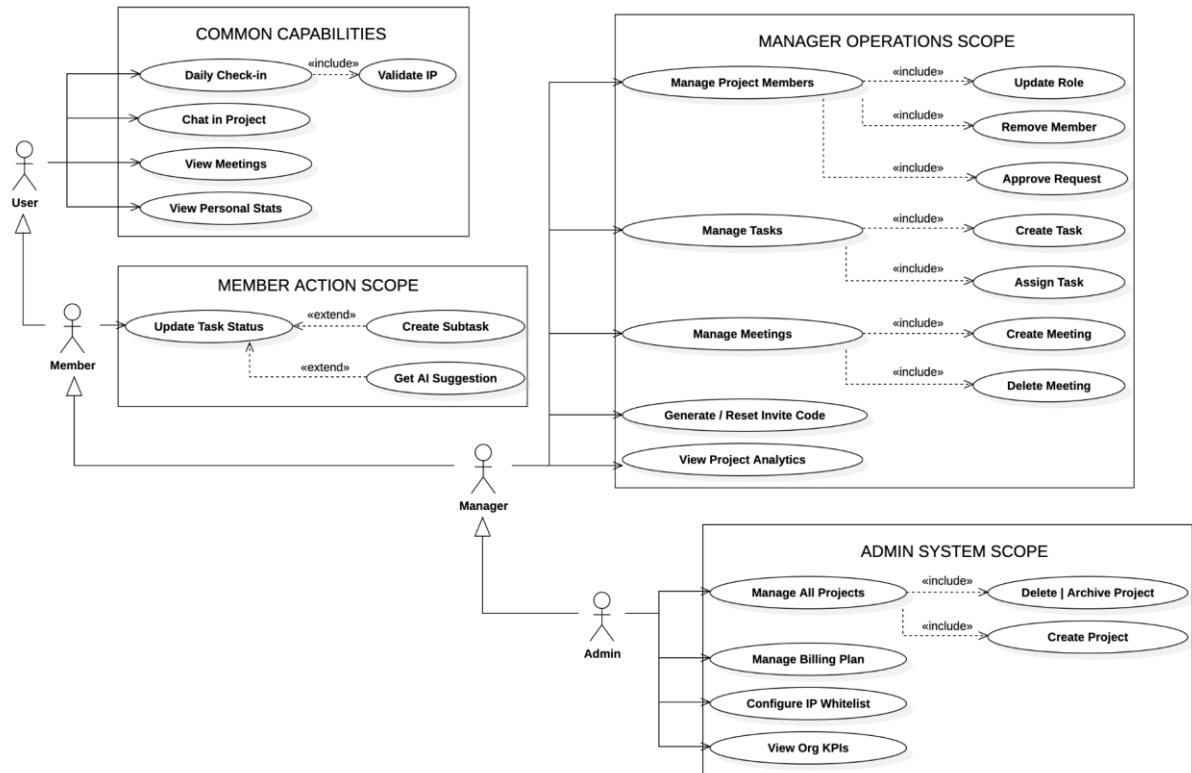


Figure 2: Role-Based Operations

This diagram illustrates the Role-Based Access Control model of the system, visualizing how permissions expand hierarchically from basic users to top-level administrators.

Key Design Logic:

- **Member:** Focuses on personal productivity. They perform daily actions like checking in, chatting, updating task status, and utilizing AI suggestions.
- **Manager:** Inherits all member capabilities but adds authority over specific projects. They act as team leaders to manage members, assign tasks, and control meeting schedules.
- **Admin:** Possesses full control over the entire organization. They handle high-level operations such as managing billing plans, configuring security (IP Whitelist), and overseeing the lifecycle of all projects.

4.3 Use Case and Scenario Description

AUTHENTICATION & ONBOARDING

4.3.1. Register & Login (Refers to SUB-FEATURE 1.1)

Brief Description

Allows users to create a new account or access the system using traditional register (email and password) or via Google OAuth 2.0 integration.

Actors

Guest User.

Pre-conditions

None.

Flow of Events

1. Basic Flows

The table below describes the process when a user authenticates via traditional login or Google OAuth.

Actor Action	System Action	Data
1. Enters Email and Password or clicks "Login with Google".	2. Receives and validates the input format.	Credentials (Email, Password) or OAuth Token
3. (None)	4. Verifies credentials against the database or processes the Google OAuth token.	User Record
5. (None)	6. Generates a stateless JSON Web Token (JWT) containing the unique UserID and System Role.	JWT Payload
7. (None)	8. Redirects the user to the Dashboard and stores the token in the client-side storage.	Authentication Token

Table 2: Login Workflow

2. Alternative Flows

(i) Invalid Credentials

If the user enters an incorrect email or password, the system displays an authentication error message and prompts them to re-enter their information.

(ii) Duplicate Email during Registration

If a user tries to register with an email that already exists in the system, the system flags a "ValidationError" and prevents account creation.

(iii) OAuth Cancellation

If the user closes the Google login popup before completion, the system returns the user to the login screen without generating a session.

Post-conditions

- The user is successfully authenticated and granted access to the system.
- The JWT is securely stored in the Client for subsequent authorized API requests.

Requirements

None

4.3.2. Join Project & Approve Request (Refers to SUB-FEATURE 1.3)

Brief Description

Allows users to join a project using a unique invite code and enables Project Managers to manage these incoming membership requests.

Actors

User (Applicant), Project Manager/Admin.

Pre-conditions

- The Project Manager must have generated a valid invite code for the project.
- The User must have received the 6-character invite code.

Flow of Events**1. Basic Flows**

The table below describes the process of joining a project via invite code and the subsequent approval by a manager.

Actor Action	System Action	Data
1. Enter the unique invite code in the onboarding interface.	2. Validates the code format and ensures the target project is not archived.	Invite Code
3. (None)	4. Creates a project membership record and assigns the user to the organization context.	ProjectMember Record
5. Manager accesses the "Pending Requests" in Project Settings.	6. Retrieves and displays all users with a pending status for the project.	Pending Member List
7. The manager clicks "Approve" to accept the user.	8. Updates member status to ACTIVE, assigns the default role, and triggers a system notification.	Updated Status (ACTIVE)

Table 3: Joining and Inviting Workflow

2. Alternative Flows**(i) Invalid or Expired Code**

In Step 2, if the code is incorrect or has been reset by the manager, the system returns an "Invalid or expired invite code" error.

(ii) Already a Member

If the user is already part of the project or has an existing request, the system prevents duplicate entries and displays a "Conflict Error".

(iii) Project Archived

If the project status is set to "ARCHIVED", the system blocks the join request to maintain data integrity.

(iv) Manager Rejects

If the manager denies the request, the pending membership record is removed, and the user remains outside the project.

Post-conditions

- The User is officially added as an active member of the project.
- The User gains full access to project-specific modules such as the Kanban board and Chat.
- A new JWT is issued to the user to include the updated organization context.

Requirements

None

CORE TASK MANAGEMENT

4.3.3. Manage Task Details (Refers to SUB-FEATURE 2.2 & 2.4)

Brief Description

This Use Case describes how Project Managers create new tasks, assign specific members, and categorize with labels.

Actors

Project Manager.

Pre-conditions

- The Project Manager is logged into the system.
- The Project Dashboard is currently active.

Flow of Events

1.Basic Flows

The table below describes the process when the Project Manager creates a new task.

Actor Action	System Action	Data
1. Click the "Add Task" button.	2. Open the "Create New Task" form.	None
3. Enters task details (Title, Priority, Deadline) and selects a member.	4. Check if the date is valid.	Task Information (Name, Due Date, Priority, Assignee)
5. Click "Create" to finish.	6. Verifies that all required info is provided. 7. Saves the new task to the system. 8. Returns a success signal.	New Task Record
9. (None)	10. Updates the screen to show the new task card immediately.	Updated Kanban Board

Table 4: Manage Task Details Basic Workflow

2.Alternative Flows

(i) Missing Information

If in the Basic Flow, at step 5, the Manager forgets to enter the Title or Deadline, the System highlights the empty fields in red and shows a message "Please fill in all required fields". The Manager must complete the information to proceed.

(ii) Cancel Creation

If the Manager clicks "Cancel" at any time, the System closes the form and no task is created.

(iii) System Error

If the System cannot save the task (e.g., server issue), it displays a message "Creation failed, please try again" and keeps the form open so the Manager doesn't lose data.

Post-conditions

- The new task is saved in the database.
- A new task appears in the "TODO" column.

Requirements

- Assignee must be an active member of the current project.
- The deadline cannot be in the past.

4.3.4. Update Task Status (Refers to SUB-FEATURE 2.1 & 2.3)

Brief description

This use case describes how users reorganize tasks on the Kanban board. It supports changing task status and reordering tasks using a drag-and-drop interface.

Actors

Can only move and reorder tasks assigned to them (Even a manager).

Pre-conditions

- The user is logged into the system.
- The user has access to the specific Project.
- The Kanban Board is successfully loaded with tasks.

Flow of events:

1. Basic Flows

The table describes the process when a User drags a task to a new position

Actor Action	System Action	Data
1. The Actor clicks and holds a Task Card, dragging it to a new column (from TODO to DONE)	2. The System calculates the new orderIndex on the client-side using the Floating-Point Indexing	- taskId - new Status - New Position Index
3. The Actor drops the card.	4.The system validates the user's permissions and updates the Task document in the Database.	- Update Request Object

Table 5: Kanban Drag & Drop Workflow

2. Alternative Flows

(i) Permission Denied (Member Restriction)

If in the Basic Flows, step 4, the Actor is a Member trying to move a task assigned to another user:

- The Backend rejects the request with a 403 Forbidden error.
- The System (Client) detects the error, reverts the Task Card to its original position (Snap-back animation), and displays a "Permission Denied" toast message.

(ii) Network Failure / Server Error

If the API call fails due to network issues:

- The System displays an error message: "Failed to update task position".

Post-conditions

- The task's status and orderIndex are updated in the MongoDB database.
- The Kanban view is synchronized for all concurrent users in the project.

Requirement:

None.

COLLABORATION

4.3.5. Real-time Chat (Refers to SUB-FEATURE 3.1)

Brief Description

This use case describes how the users can exchange instant text messages securely within a specific project room.

Actors

Member, Project Manager, Admin.

Pre-conditions

- The user is logged in and verified (Valid Token).
- The user has opened the project chat interface.

Flow of Events

1.Basic Flows

The table below describes the process of sending and receiving a chat message in real-time.

Actor Action	System Action	Data
1. Types a message in the input box and presses Enter (or clicks Send).	2. Checks if the message content is not empty.	Message Content
3. (None)	4. Receives the message via a secure channel. 5. Saves the message to the database permanently. 6. Broadcasts the message to everyone in the project room immediately.	New Message Record (Content + Sender Info)
7. (All Members in Room)	8. Receives the signal and appends the new message bubble to the chat window (bottom of the list).	Live Chat Update

Table 6: Real-time Chat Basic Workflow

2.Alternative Flows

(i) Empty Message

If in the Basic Flow, at step 1, the Actor tries to send a message containing only spaces, the System ignores the action. No data is sent to the server.

(ii) Not Joined Room (Connection Error)

If in the Basic Flow, at step 3, the System detects the User has not joined the project room, the message is not sent. The System attempts to reconnect and displays a "Reconnecting..." status.

(iii) Unauthorized Access

If a User tries to send a message but their Token is expired or invalid (checked by Middleware), the System rejects the connection and forces the User to log in again.

Post-conditions

- The message is saved in the chat history for future loading.
- All online members see the new message instantly.

Requirements

- The latency from sending to appearing on other screens should be minimal.
- Each message must clearly display the Sender's Name and Avatar.

4.3.6. Comment & Mention (Refers to SUB-FEATURE 3.2)**Brief Description**

This use case describes how the project members discuss specific tasks and tag other members (using "@") to notify them immediately.

Actors

Member, Project Manager.

Pre-conditions

- The user is logged in.
- The user is viewing the Task Details modal.

Flow of Events**1.Basic Flows**

The table below describes the process of posting a comment and notifying mentioned users.

Actor Action	System Action	Data
1. Type a comment in the text box (e.g., "Check this @Hieu") and click "Send".	2. Validates that the comment content is not empty.	Comment Content
3. (None)	4. Save the comment to the database. 5. Detects the "@Huy" tag and creates a notification record. 6. Instantly sends an alert to User "Huy". 7. Logs this activity to the project history.	New Comment Record & Mention Alert
8. (Current User)	9. Updates the comment list to show the new message immediately.	Updated Discussion List
10. (Tagged User - "Hieu")	11. Receives a bell notification: "Admin mentioned you in task...".	New Notification

Table 7: Comment & Mention Basic Workflow

2. Alternative Flows**(i) Empty Content**

If in the Basic Flow, at step 1, the User tries to send a comment without text, the System prevents the action and displays a warning: "Content is required".

(ii) Unauthorized Access

If a User tries to comment on a project they are not a member of (e.g., via a direct link hack), the System checks permissions and returns an "Access Denied" error.

(iii) Task Not Found

If the task has been deleted by another manager just before the comment is sent, the System displays an error "Task not found" and refreshes the page.

Post-conditions

- The comment is permanently saved in the task.
- The tagged user has an unread notification in their inbox.

Requirements

- The mentioned user must receive the alert.
- The system must record that "User X commented on Task Y" for audit purposes.

4.3.7. Schedule Meeting (Refers to SUB-FEATURE 3.3)**Brief Description**

This use case describes how Project Managers and Admins can book new meetings while automatically preventing schedule conflicts.

Actors

Project Manager, Admin.

Pre-conditions

- The user is logged in with management rights.
- The user is viewing the Project Calendar or Meeting List.

Flow of Events**1.Basic Flows**

The table below describes the process of scheduling a meeting without any time conflicts.

Actor Action	System Action	Data
1. Clicks "New Meeting" and fills in Title, Date, Start Time, End Time, and Link.	2. Check if the End Time is after the Start Time.	Meeting Details
3. Click "Create Schedule".	4. Scans existing meetings to ensure no overlap occurs in the selected time slot. 5. Saves the meeting to the schedule. 6. Send notifications to all project members.	Time Slot Availability & New Meeting Record
7. (None)	8. Updates the UI to show the new event on the timeline.	Updated Calendar
9. (Project Members)	10. Receives a notification: "New meeting scheduled: [Title]".	Invitation Alert

Table 8: Schedule Meeting Basic Workflow

2. Alternative Flows

(i) Time Conflict

If in the Basic Flow, at step 3, the System detects that another meeting is already scheduled during the chosen time frame (e.g., 9:00 - 10:00 is taken), it prevents creation and displays an error: "Time slot conflict! Please choose another time.". The Manager must adjust the time to proceed.

(ii) Invalid Time Range

If in the Basic Flow, at step 1, the Actor sets the End Time earlier than the Start Time (e.g., Start 10:00, End 09:00), the System blocks the submission and highlights the error.

(iii) Cancellation

If the Actor clicks "Cancel" while filling out the form, the System discards the input and returns to the Meeting List.

Post-conditions

- The meeting is officially recorded in the Project Schedule.
- All members are notified of the upcoming event.

Requirements

- Two meetings cannot occur simultaneously within the same project.
- Once created, the meeting details must be visible to all members immediately.

AI PRODUCTIVITY

4.3.8. Generate AI Subtasks (Refers to SUB-FEATURE 6.1)

Brief description

This use case describes how the system utilizes (Groq Cloud/Llama 3) to analyze a task's title and description, automatically breaking it down into a checklist of actionable subtasks.

Pre-conditions

- The user is logged in and viewing the Task Details modal.
- The task must have a valid Title (and optionally a Description).

Flow of event**1. Basic Flow**

The following table describes the process when a User requests AI assistance to break down a task.

Actor Action	System Action	Data
1. The Actor clicks the "Magic Subtask" button inside the Task Detail view.	2. The System constructs a structured prompt combining the task details and sends a request to the Groq Cloud API.	Task Title and Description
(None)	3. The System receives the raw string response and parses it into a structured list.	JSON Array of Subtasks
	4. The System saves the new subtasks to the database and updates the checklist UI.	List of suggested subtasks

Table 9: AI Subtask Generation Workflow

2. Alternative Flows

(i) AI Service Unavailable

If the Groq API times out or returns a 500 error, the System aborts the process and displays a toast message: "AI Service is currently busy."

(ii) Invalid AI Response

If the AI returns unstructured text (invalid JSON), the System logs the error and alerts the user: "Could not generate structured subtasks."

(iii) Empty Context

If the task has no description, the System proceeds with the Title only but may prompt the user to add more details for better results.

Post-conditions

- The task checklist is populated with new items, and the progress bar is initialized.

Requirements

- The AI response must be strictly formatted as a JSON Array to ensure correct parsing by the backend.

MONITORING & ATTENDANCE

4.3.9. Attendance Check-in (Refers to SUB-FEATURE 4.3)

Brief description

This use case describes how a member performs a daily attendance check-in. The system enforces security by validating the client's IP address against the Organization's Whitelist before recording the timestamp.

Actors

Member

Pre-conditions

- The user is logged in and belongs to an Organization.
- The user has not checked in for the current day (System enforces one check-in per day).
- The user is physically connected to the office's network.

Flow of events

1. Basic Flow

The table below describes the successful check-in process.

Actor Action	System Action	Data
1. The Actor clicks the "Check-in" button on the Attendance screen.	2. The System detects the device's public IP address and normalizes it for comparison.	Current Network IP
(None)	3. The System verifies if this IP matches the Organization's allowed list.	Whitelisted IPs

(None)	4. The System compares the current time with the standard start time to mark the status as Present or Late.	Current Time
(None)	5. The System saves the attendance record and displays a success notification.	Status (Present/Late)

Table 10: IP-Based Attendance Check-in Workflow

2. Alternative Flows

(i) Invalid Network (IP Mismatch)

If the detected IP is not in the whitelist:

- The System blocks the request (403 Forbidden).
- The System displays an error: "Invalid IP. Please connect to the company network."

(ii) Duplicate

Check-in If the user has already checked in today:

- The System prevents the action.
- The System alerts: "You have already checked in today."

(iii) No Organization

If the user account is not linked to any active Organization:

- The System denies the request and prompts the user to join an organization first.

Post-conditions

- A new attendance record is created, and the user's daily schedule is displayed.

Requirements

- The system must handle both IPv4 and IPv6 formats.
- The validation logic must support "Wildcard IPs" (e.g., 192.168.1.) to allow dynamic office subnets.

4.3.10. Use case: View Attendance Reports (SUB-FEATURE 4.2)

Brief description

This use case describes how Managers and Admins review employee attendance history. The system records check-in logs into a report (Present, Late, Absent counts) with filterings.

Actors:

- Admin: Can view reports for the entire organization or filter by any project.
- Project Manager: Can view reports strictly for members within their assigned projects.

Pre-conditions:

- The user has Management rights (Admin/Manager role).
- The user is accessing the dashboard within a valid Organization context.

Flow of events

1. Basic Flow

The table below describes the process of generating an attendance report.

Actor Action	System Action	Data
1. The Actor navigates to the "Attendance" tab and selects a Date Range.	2. The System validates the Actor's permissions and retrieves the list of target members.	- Selected Project - Start/End Date
(None)	3. The System queries attendance logs and calculates statistics (Total Present, Late count, Absent count) for each member.	- Attendance Logs
(None)	4. The System identifies specific dates for "Late" and "Absent" records to provide detailed insights.	- Late/Absent Details
(None)	5. The System renders a data table displaying the status summary for each employee.	- Attendance Stats

Table 11: Attendance Report Generation Workflow

2. Alternative Flows

(i) No Data Found

If no attendance records match the selected filters:

- The System returns an empty list.
- The UI displays: "No attendance records found for this period."

(ii) Invalid Date Range

If the Actor selects a Start Date after the End Date:

- The System validates the input and shows an error: "Invalid date range."
- The data table is not updated.

Post-conditions

The Manager/Admin views the attendance performance of the team.

Requirements

- Data Aggregation: The system must automatically calculate Present, Late, and Absent totals based on the Organization's standard check-in hour.
- Scope Isolation: Managers must not be able to see attendance data of users outside their assigned projects.

4.3.11. View Dashboard & Analytics (Refers to SUB-FEATURE 5.2)

Brief Description

This use case describes the central hub for users to view performance metrics, workload statistics, and project health visuals based on their role.

Actors

Admin, Project Manager, Member.

Pre-conditions

- The user is logged in.
- The user belongs to at least one Organization or Project.

Flow of Events

1. Basic Flows

The table below describes the process of viewing the Dashboard and filtering data.

Actor Action	System Action	Data
1. Click the "Dashboard" menu item.	1. Detects the User's Role (Admin, Manager, or Member). 2. Aggregates data from the database for the current month. 3. Displays the appropriate layout.	Role Context
2. (Member View)	Shows "My Pending Tasks", "Overdue Alerts", and "Personal Completion Rate".	Personal KPIs
3. (Manager/Admin View)	Shows "Project Health", "Member Workload", and "Task Status Distribution" charts.	Team Performance Metrics
4. Selects a specific Month and Year from the filter dropdown (e.g., "October 2025").	1. Validates the selected date range. 2. Recalculates all numbers and redraws the charts for that specific period.	Filtered Statistics

Table 12: View Dashboard & Analytics Basic Workflow

2. Alternative Flows

(i) Invalid Date Filter

If in the Basic Flow, at step 4, the User manually enters an invalid month (e.g., Month 13), the System ignores the request and keeps displaying the current month's data, showing a warning: "Invalid month selection".

(ii) No Data Available (Empty State)

If a new project has just been created and has no tasks yet, the System displays all counters as "0" and shows an empty placeholder image for charts instead of breaking the layout.

(iii) Organization Missing

If an Admin tries to view stats but is not currently switched into any Organization context, the System prompts: "Please select an Organization to view analytics".

Post-conditions

- The User sees the real-time health check of their work/project.
- No data is modified (Read-only view).

Requirements

- Critical numbers (Overdue, Todo) must be highlighted in colors (Red/Yellow) for quick attention.
- Dashboard loading time must be under 1 second even with large datasets.

PROJECT & USER ADMINISTRATION

4.3.12. Manage Project Members (Refers to SUB-FEATURE 4.4)

Brief Description

Allows authorized users to manage project membership by assigning roles, updating member permissions, or removing users from the project environment.

Actors

- Admin: Has full control over all project members.
- Project Manager: Has restricted management permissions.

Pre-conditions

- The actor must have a valid session and specific management permissions within the targeted project.
- The member list must be active and accessible within the organization context.

Flow of Events

1. Basic Flows

The table below describes the administrative process for managing project memberships.

Actor Action	System Action	Data
1. Navigates to the "Members" tab within the Project Settings interface.	2. Retrieves the list of project members and their current roles from the database.	Member List
3. Admin updates a user's role (e.g., from Member to Manager).	4. Validates the request and updates the roleInProject field in the collection.	Updated Role
5. Admin or Manager clicks the "Remove" button for a specific member.	6. Checks role hierarchy and permission levels before processing the removal.	Target Member ID
7. (None)	8. Updates the ProjectMember collection, removes access, and emits a socket event to force logout the removed user.	Removal Signal

Table 13: Project Managing Workflow

2. Alternative Flows

(i) Manager Attempting Role Update

If a Project Manager attempts to change a member's role, the system detects a lack of specific authorization and blocks the request.

(ii) Self-Removal Restriction

Users are prohibited from removing themselves from a project to prevent accidental loss of access.

(iii) Project Creator Protection

The system prevents the removal of the project creator/owner to maintain administrative integrity.

Post-conditions

- The ProjectMember collection is updated to reflect the new roles or membership status.
- The UI is immediately updated to show the current member list.
- Removed users lose all access to project resources, including the Kanban board and Chat.

Requirements

- The system must validate the organization's current project count against the active subscription plan before initialization.
- Project Managers are prohibited from updating user roles or removing other Managers and Admins.

4.3.13. Manage Projects (Admin CRUD) (Refers to SUB-FEATURE 5.1)**Brief Description**

Allows Organization Admins to perform high-level management of the project lifecycle, including creation with plan-based validation, detailed monitoring, and secure removal.

Actors

Admin.

Pre-conditions

- The Admin must be authenticated and have an active organization context.
- The organization must be assigned a service plan (Free or Premium) to determine project limits.

Flow of Events**1. Basic Flows**

The table below describes the administrative process for creating, viewing, and deleting projects within an organization.

Actor Action	System Action	Data
1. Accesses the project management dashboard.	2. Displays the list of all active projects.	Project List
3. Enters project details (Name, Deadline).	4. Checks if the organization has reached its project limit.	Project Metadata
5. Clicks "Create" to finish.	6. Saves the project and assigns the Admin as the owner.	New Project Record
7. Selects a project to view more information.	8. Shows project statistics and a summary of recent activities.	Project Summary
9. Selects the option to delete a project.	10. Hides the project from the active dashboard.	Updated Status

Table 14: Project management Workflow

2. Alternative Flows**(i) Plan Limit Reached**

If an Admin on a Free plan attempts to create more than one project, the system blocks the request and returns a PLAN_LIMIT_REACHED error.

(ii) Invalid Project ID

If the Admin attempts to access or delete a project with an incorrect ID format, the system returns a `ValidationError`.

(iii) Unauthorized Access

If an Admin attempts to manage a project belonging to a different organization, the system blocks the request to ensure strict data isolation.

Post-conditions

- A new project is successfully initialized and accessible to invited members.
- Deleted projects are hidden from the active project list while maintaining a `deletedAt` timestamp for audit integrity.
- System resources and quotas are updated according to the current organizational plan.

Requirements

None

ORGANIZATION CONFIGURATION

4.3.14. Manage Subscription (Billing) (Refers to SUB-FEATURE 1.2)

Brief Description

This use case describes how the Organization Admin upgrades to Premium, manages payment methods, or cancels subscriptions securely via Stripe Integration.

Actors

Admin.

Pre-conditions

- The user is logged in as an Admin of the Organization.
- The user is viewing the "Billing & Settings" tab.

Flow of Events**1.Basic Flows**

The table below details the secure process of upgrading an Organization's subscription plan to Premium via the Stripe Payment Gateway.

Actor Action	System Action	Data
1. Click the "Upgrade to Premium" button.	2. Connects to Stripe Payment Gateway. 3. Generates a secure session link. 4. Redirects the Admin to the Stripe Checkout page.	Secure Payment Link
5. (On Stripe Page) Enters credit card details and confirms payment.	6. Processes the transaction securely.	Payment Credentials (Handled by Stripe)
7. (Stripe Redirects Back)	8. Verifies the successful transaction. 9. Updates the Organization's plan to "PREMIUM" immediately. 10. Unlocks unlimited projects and AI	Updated Subscription Status

	features.	
11. (None)	12. Displays a "Premium Active" badge and sends a confirmation email.	Success Notification

Table 15: Manage Subscription Basic Workflow

2. Alternative Flows

(i) Manage or Cancel Subscription

If the Admin clicks "Manage Billing", the System generates a portal link and redirects them to the Stripe Customer Portal. There, the Admin can update credit cards or cancel the plan. The changes are automatically synced back to the system.

(ii) Resume Subscription

If the Admin previously canceled but the plan hasn't expired yet, they can click "Resume Subscription". The System immediately reactivates auto-renewal and confirms via a notification: "Subscription Resumed".

(iii) Payment Cancellation

If in Step 5, the Admin clicks "Back" or "Cancel" on the Stripe page, the System redirects them back to the Billing page with a message: "Payment process was cancelled". No charges are made.

(iv) Invalid Plan Selection

If the Admin tries to select a plan that doesn't exist (e.g., via API manipulation), the System returns an error "Invalid plan name" and prevents the checkout process.

Post-conditions

- The Organization's plan is updated (Free ↔ Premium).
- Quotas (Project limit, AI usage) are adjusted accordingly.

Requirements

- Credit card information must NEVER be stored on the application server; it is handled entirely by Stripe.
- The system must handle Webhook events to ensure the plan status matches the actual payment status.

4.3.15. Manage IP Whitelist (Refers to SUB-FEATURE 7.2)

Brief description

This use case describes how the Organization Admin configures the network security by adding or removing IP addresses to restrict attendance check-ins.

Actors

Admin

Pre-conditions

The user is logged in as an Admin of the Organization.

Flow of events

1. Basic Flow

The table below describes the process of adding a new trusted IP address.

Actor Action	System Action	Data
1. The Admin in Attendance Settings and clicks "Add New IP".	2. The System displays the input form.	
3. The Admin enters the IP details and clicks "Save".	4. The System validates the format (IPv4 or Wildcard) and checks for duplicates.	- IP Address - Location Description
	5. The System saves the IP to the whitelist and updates the access rules immediately.	- Status (Active)
	6. The System shows a success message and refreshes the list.	

Table 16: IP Whitelist Configuration Workflow

2. Alternative Flows

(i) Invalid IP Format

If the Admin enters an incorrect format:

- The System blocks the request.
- The System displays an error: "Invalid IP format."

(ii) Duplicate IP Address

If the entered IP is already in the whitelist:

- The System prevents saving.
- The System alerts: "This IP address already exists."

(iii) Remove IP Address

If the Admin clicks "Delete" on an existing IP:

- The System removes the IP from the database.
- Access from that IP is immediately revoked.

Post-conditions

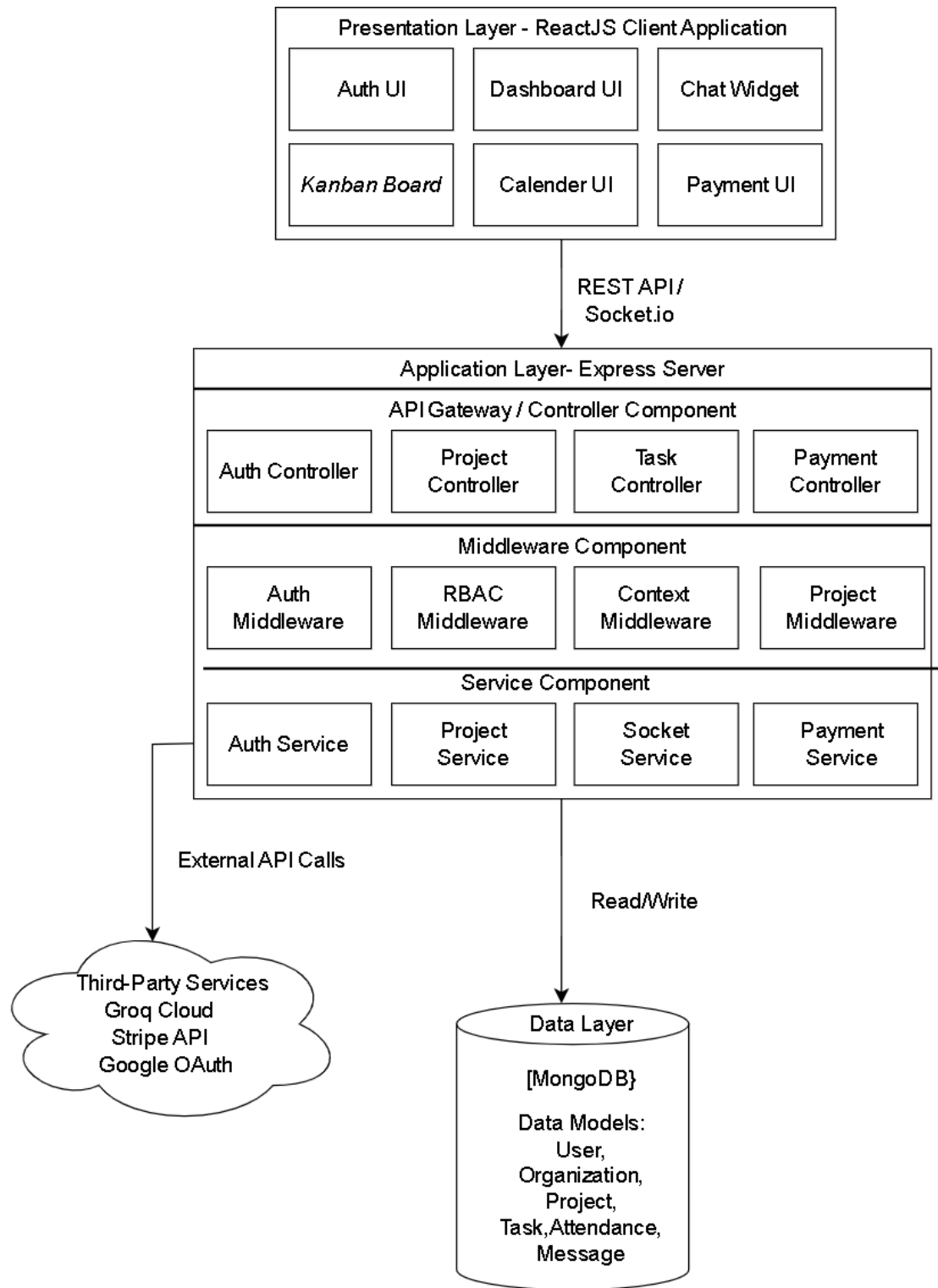
The Organization's whitelist is updated. Attendance requests are filtered based on the new configuration.

Requirements

- The system must record Admin added the IP for security tracking.

5. METHODOLOGY

5.1 System Architecture



Figure

3: High-Level System Component Architecture

This is the System Architecture Diagram for the Web Application for Organizational Project Team Management. The system is divided into four distinct layers: Presentation Layer

(Client), Application Layer (Backend), Data Layer (Database). and Third-Party Services. Communication between the Client and Backend is controlled via RESTful HTTP requests while WebSocket (Socket.io) protocols are utilized to enable low-latency, real-time features.

5.2. Database Design

5.2.1. Entity Relationship Diagram (ERD)

The system uses MongoDB to cover the flexibility and scalability of a SaaS application. The database schema is designed around the Multi-tenancy architecture, where the Organization collection is the root entity for all data isolation logic.

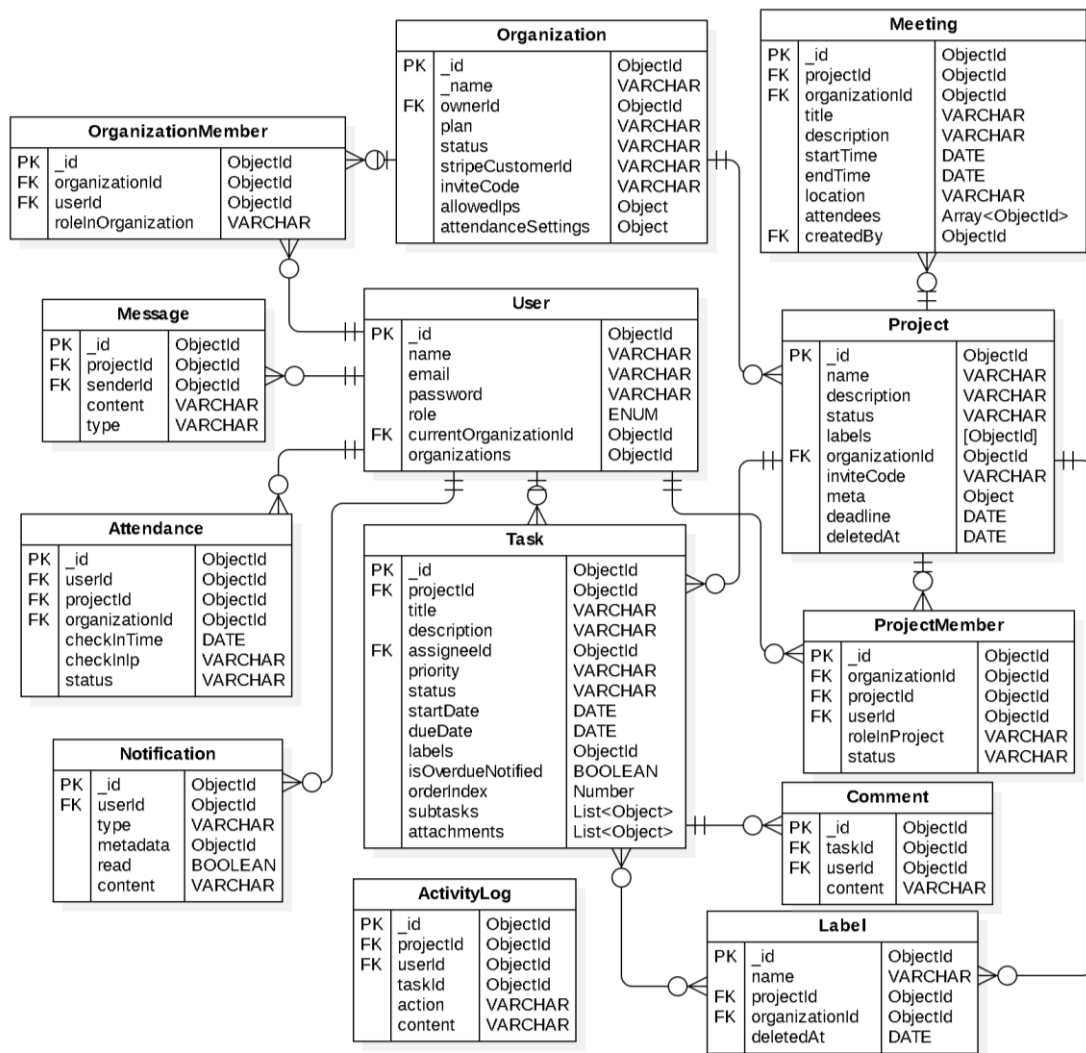


Figure 4: System Entity Relationship Diagram (ERD)

5.2.2. Schema Analysis

5.2.2.1. Organization Collection

Organization		
PK	_id	ObjectId
	_name	VARCHAR
FK	ownerId	ObjectId
	plan	VARCHAR
	status	VARCHAR
	stripeCustomerId	VARCHAR
	inviteCode	VARCHAR
	allowedIps	Object
	attendanceSettings	Object

Figure 5: Database Organization

- This collection serves as the root entity for the multi-tenancy architecture.
- The “id” field acts as Primary key, each Organization has their unique data and information. The “invite code” shows that users can join the Organization/Project via a code.
- The “ownerId” represents the user who is an administrator of the Organization.
- The “plan” sticks with the ownerId to show whether the Organization is FREE/PREMIUM.
- The “allowedIps” indicates the whitelisted IP addresses for security checks.

5.2.2.2. OrganizationMember Collection

OrganizationMember		
PK	_id	ObjectId
FK	organizationId	ObjectId
FK	userId	ObjectId
	roleInOrganization	VARCHAR

Figure 6: Database OrganizationMember

- This collection manages the membership relationship between Users and Organizations. The “userId” shows each user belonging to the Organization.
- The “roleInOrganization” indicates the role in Organization (ADMIN/MEMBER).

5.2.2.3. User Collection

User		
PK	_id	ObjectId
	name	VARCHAR
	email	VARCHAR
	password	VARCHAR
	role	ENUM
FK	currentOrganizationId	ObjectId
	organizations	ObjectId

Figure 7: Database User

- This collection represents the global identity of a person in the system, it includes currentOrganizationID to show the current Organization of the user. Moreover, “organizations” indicates that users can join multiple Organizations. However, we currently limit that they just only join one Organization to minimize the logic part and scope.

5.2.2.4. Project Collection

Project		
PK	<u>_id</u>	ObjectId
	name	VARCHAR
	description	VARCHAR
	status	VARCHAR
	labels	[ObjectId]
FK	organizationId	ObjectId
	inviteCode	VARCHAR
	meta	Object
	deadline	DATE
	deletedAt	DATE

Figure 8: Database Project

- This collection acts as a container for tasks and resources within an organization. The “organizationId” shows the project belonging to which Organization.
- The “inviteCode” is a unique code to let users join a project. Moreover, the “status” indicates whether the project is “ACTIVE/ARCHIVED”. Each project also has their own “label” to show specific or short description groups of tasks.

5.2.2.5. ProjectMember Collection

ProjectMember		
PK	<u>_id</u>	ObjectId
FK	organizationId	ObjectId
FK	projectId	ObjectId
FK	userId	ObjectId
	roleInProject	VARCHAR
	status	VARCHAR

Figure 9: Database ProjectMember

- This collection handles access control at the project level. It links a userId to a projectId with a specific role (Manager or Admin). The “Status” show whether user active or not
- User can join multiple projects within the Organization, via the “projectId” and “organizationId”

5.2.2.6. Task Collection

Task		
PK	<u>_id</u>	ObjectId
FK	projectId	ObjectId
	title	VARCHAR
	description	VARCHAR
FK	assigneeId	ObjectId
	priority	VARCHAR
	status	VARCHAR
	startDate	DATE
	dueDate	DATE
	labels	ObjectId
	isOverdueNotified	BOOLEAN
	orderIndex	Number
	subtasks	List<Object>
	attachments	List<Object>

Figure 10: Database Task

- This collection is the core unit of work in the system. The “priority” used to track the Kanban status (TODO, DOING, DONE) and orderIndex (a floating-point value) to efficiently manage drag-and-drop positioning without re-indexing the entire list. Subtasks are stored as an embedded array within the document.

- “startDate” and “dueDate” shows whenever the task is created and ended. It also shows the remaining hours and notify if some task is overdue.

5.2.2.7. Attendance Collection

Attendance		
PK	_id	ObjectId
FK	userId	ObjectId
FK	projectId	ObjectId
FK	organizationId	ObjectId
	checkInTime	DATE
	checkInIp	VARCHAR
	status	VARCHAR

Figure 11: Database Attendance

- This collection maintains references to User, Project, and Organization entities, enabling granular reporting at both the individual and organizational levels. It includes a status field (e.g., PRESENT, LATE) that categorizes the check-in based on the organization's standard hours.
- The userId and checkInTime are implemented to optimize the retrieval performance of historical attendance logs.

5.2.2.8. Meeting Collection

Meeting		
PK	_id	ObjectId
FK	projectId	ObjectId
FK	organizationId	ObjectId
	title	VARCHAR
	description	VARCHAR
	startTime	DATE
	endTime	DATE
	location	VARCHAR
	attendees	Array<ObjectId>
FK	createdBy	ObjectId

Figure 12: Database Meeting

- This collection facilitates event management by linking schedules to specific Project and Organization contexts, while tracking participants via the attendees array. It relies on startTime and endTime fields to avoid overlapping schedules and conflicts.

5.2.2.9. Notification Collection

Notification		
PK	_id	ObjectId
FK	userId	ObjectId
	type	VARCHAR
	metadata	ObjectId
	read	BOOLEAN
	content	VARCHAR

Figure 13: Database Notification

- Stores the history of alerts for users. The type field distinguishes between events (Assigned Task, Tagging, Meeting schedules), while metadata stores relevant IDs (like taskId) to enable deep linking—allowing the user to click the notification and be redirected immediately to the relevant context.

5.2.2.10. Message Collection

Message		
PK	_id	ObjectId
FK	projectId	ObjectId
FK	senderId	ObjectId
	content	VARCHAR
	type	VARCHAR

Figure 14: Message Notification

- This collection stores real-time communication logs within a project. It links to projectId to categorize conversations by workspace and senderId to identify the user. The type field distinguishes between text messages, images, or file attachments.

5.2.2.11. ActivityLog Collection

ActivityLog		
PK	_id	ObjectId
FK	projectId	ObjectId
FK	userId	ObjectId
	taskId	ObjectId
	action	VARCHAR
	content	VARCHAR

Figure 15: ActivityLogNotification

- This collection serves as an audit trail for the system, recording critical actions (e.g., 'User A moved Task B to Done'). It references taskId and projectId to allow filtering history logs at both the task and project levels.

5.2.2.12. Comment Collection

Comment		
PK	_id	ObjectId
FK	taskId	ObjectId
FK	userId	ObjectId
	content	VARCHAR

Figure 16: Database Comment

- This collection facilitates discussions specific to a task. It enables users to communicate context, updates, or feedback directly on a work item, linking via taskId and userId.

5.2.2.13. Label Collection

Label		
PK	_id	ObjectId
	name	VARCHAR
FK	projectId	ObjectId
FK	organizationId	ObjectId
	deletedAt	DATE

Figure 17: Database Label

- This collection manages categorization tags (e.g., 'Urgent', 'Bug', 'Feature') within a project. Labels are defined at the project level (projectId) and can be assigned to multiple tasks to support filtering and visual organization on the Kanban board.

5.3. Backend Implementation

5.3.1. Authentication & Onboarding Flow

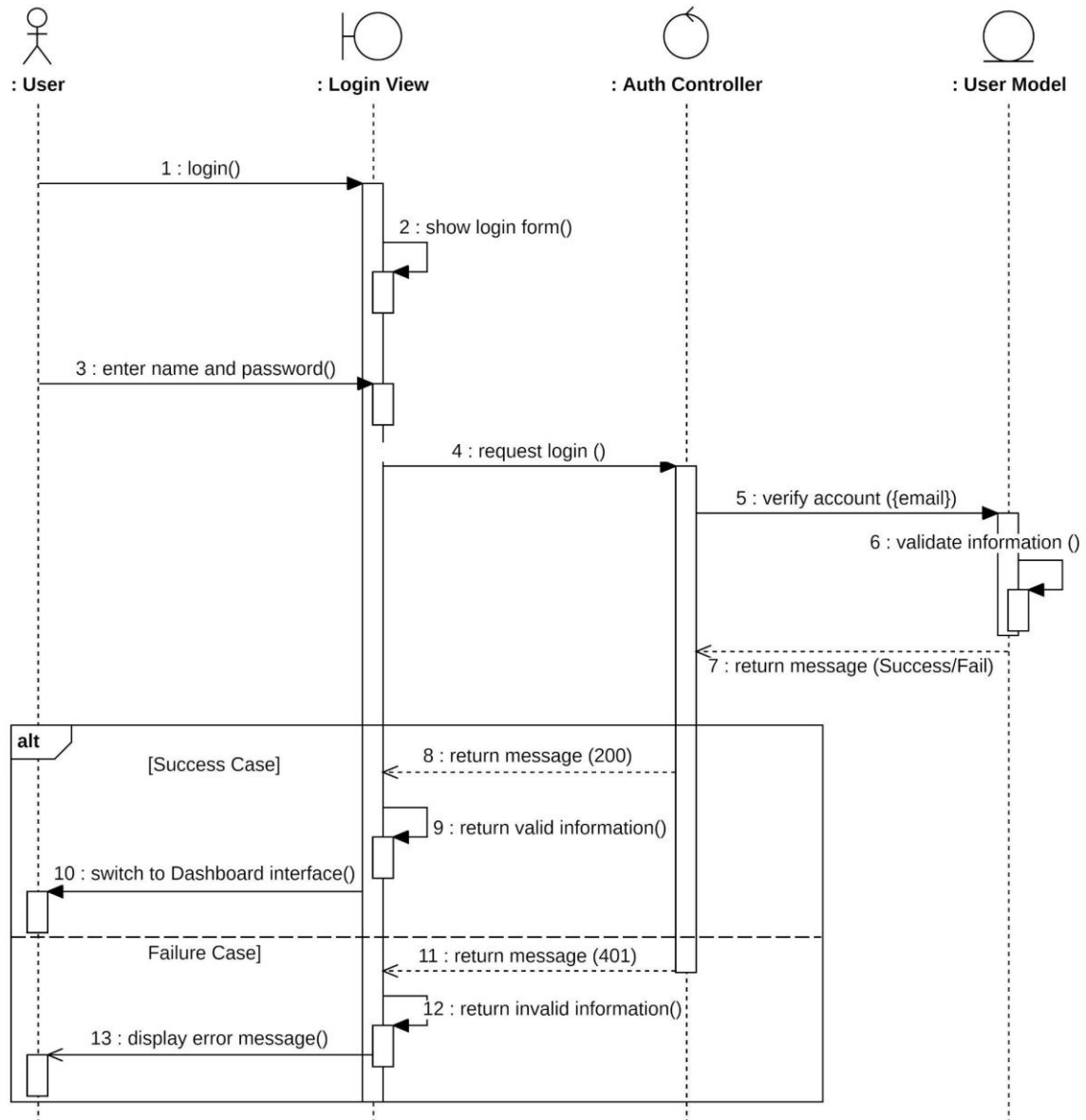


Figure 18: User Login Sequence Diagram

This sequence diagram describes how a user authenticates with the system to gain access to their workspace. The process involves the interaction between the User, the Login View, the Auth Controller, and the User Model. When a user initiates the login process, the system prompts the actor to enter their name and password through the interface. Once the credentials are submitted, the Auth Controller verifies the account details against the User Model. If the information is valid, the system logs the user in and redirects them to the Dashboard interface.

Otherwise, the system displays an error message and requires the user to re-enter their information.

5.3.2. Role-Based Access Control (RBAC) & User Management

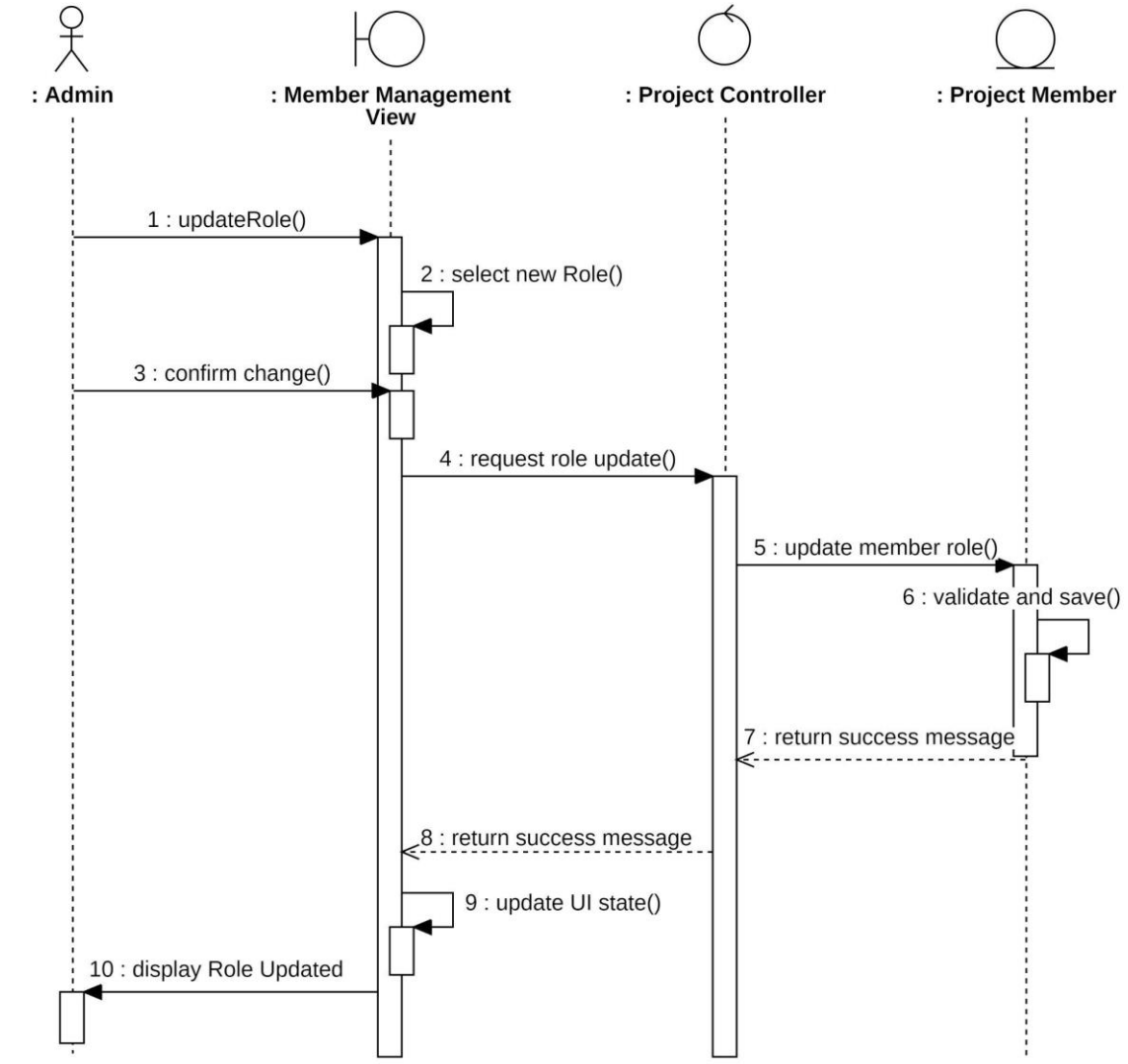


Figure 19: Member Role Update Sequence Diagram

This sequence diagram describes how an Admin updates a member's role within the system. This user must be an Admin with the required permissions to modify project membership. When an Admin wants to update a role, the system requests the actor to select a new role and confirm the change through the Member Management View. Once the Admin confirms the change, the system will check if the request is authorized and saves the updated role information.

5.3.3 Project Context Switching

In a multi-project environment, users need to flexibly switch between different workspaces. Our system provides a Project Switcher in the sidebar, enabling users to instantly toggle their working context.

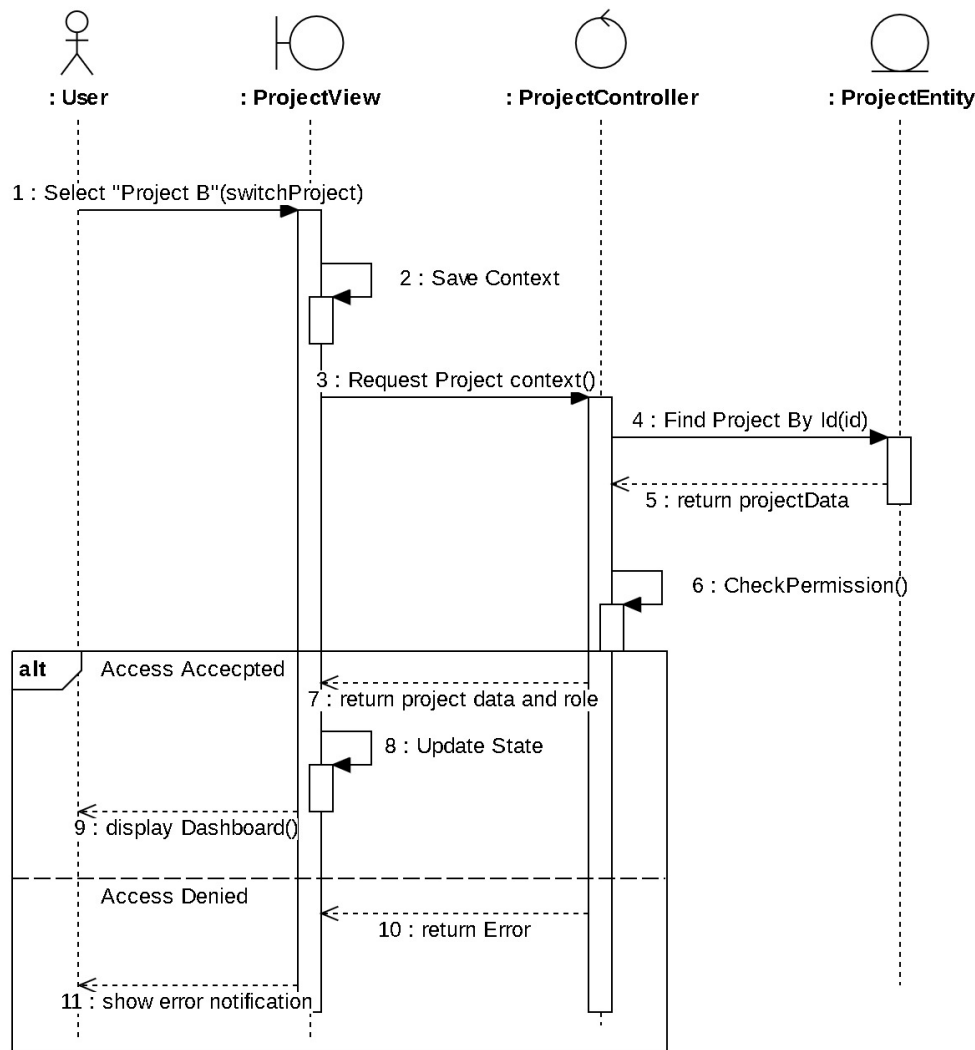


Figure 20: Project Context Switching Sequence Diagram

Figure illustrates the process when a user switches from Project A to Project B. We utilized React Context with LocalStorage in order to remember the user’s choice. This ensures and unifies the selected project by user. Security is enforced at the request level, the Backend Middleware validates the membership whenever the Frontend requests data for a new project.

5.3.4. Dashboard Analytics

To ensure a focused user experience, our system implements a Role-Based Dashboard that dynamically adapts content to the user's specific responsibilities. As illustrated in the figure below, the Controller validates the user's role to determine the appropriate data retrieval logic—fetching personal tasks for Members, project metrics for Managers, or organization KPIs for Admins. This mechanism guarantees that users receive only the most relevant insights, preventing information overload while optimizing system performance.

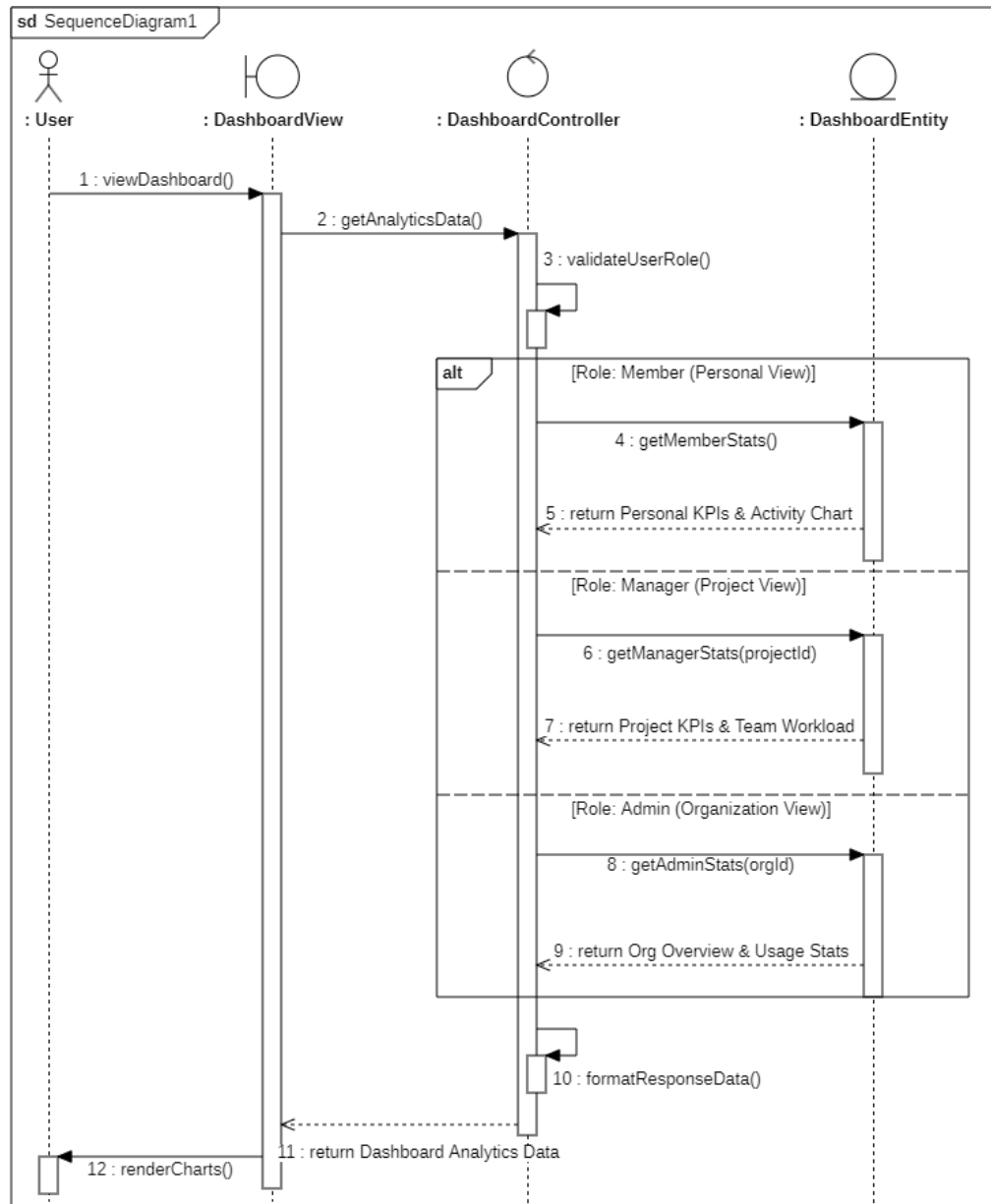


Figure 21: Dashboard Analytics & Data Aggregation Sequence Diagram

5.3.5. Invitation & Join System

Our system allows users to join projects quickly using a unique 6-digit invite code. As shown in the figure below, the Controller first verifies the code with the database; if valid, it adds the user as a "Pending" member and immediately sends a notification to the Project Manager. If the code is incorrect or expired, the system stops the process and shows an error. This ensures that new members can request access easily but still require approval before joining the team.

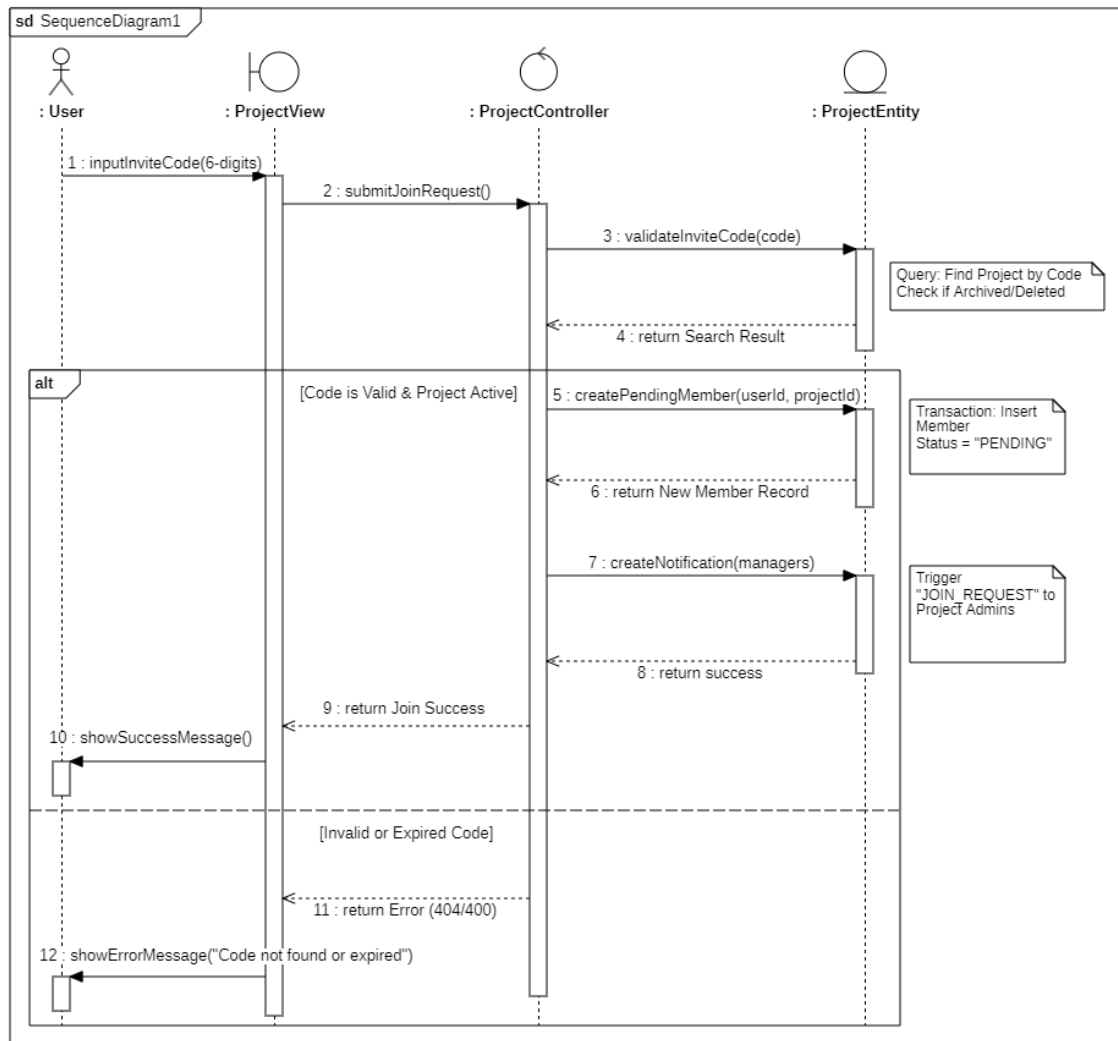


Figure 22: Project Join Request via Invite Code Sequence Diagram

5.3.6. Payment Integration

To unlock unlimited resources, the system manages plan upgrades securely through Stripe. As illustrated in the figure below, once an Admin completes the payment, Stripe sends an automatic webhook signal to our server. The Controller first verifies the request's digital signature to ensure security; upon validation, it immediately updates the organization's plan to "Premium" in the database, instantly removing the project creation limit without manual intervention.

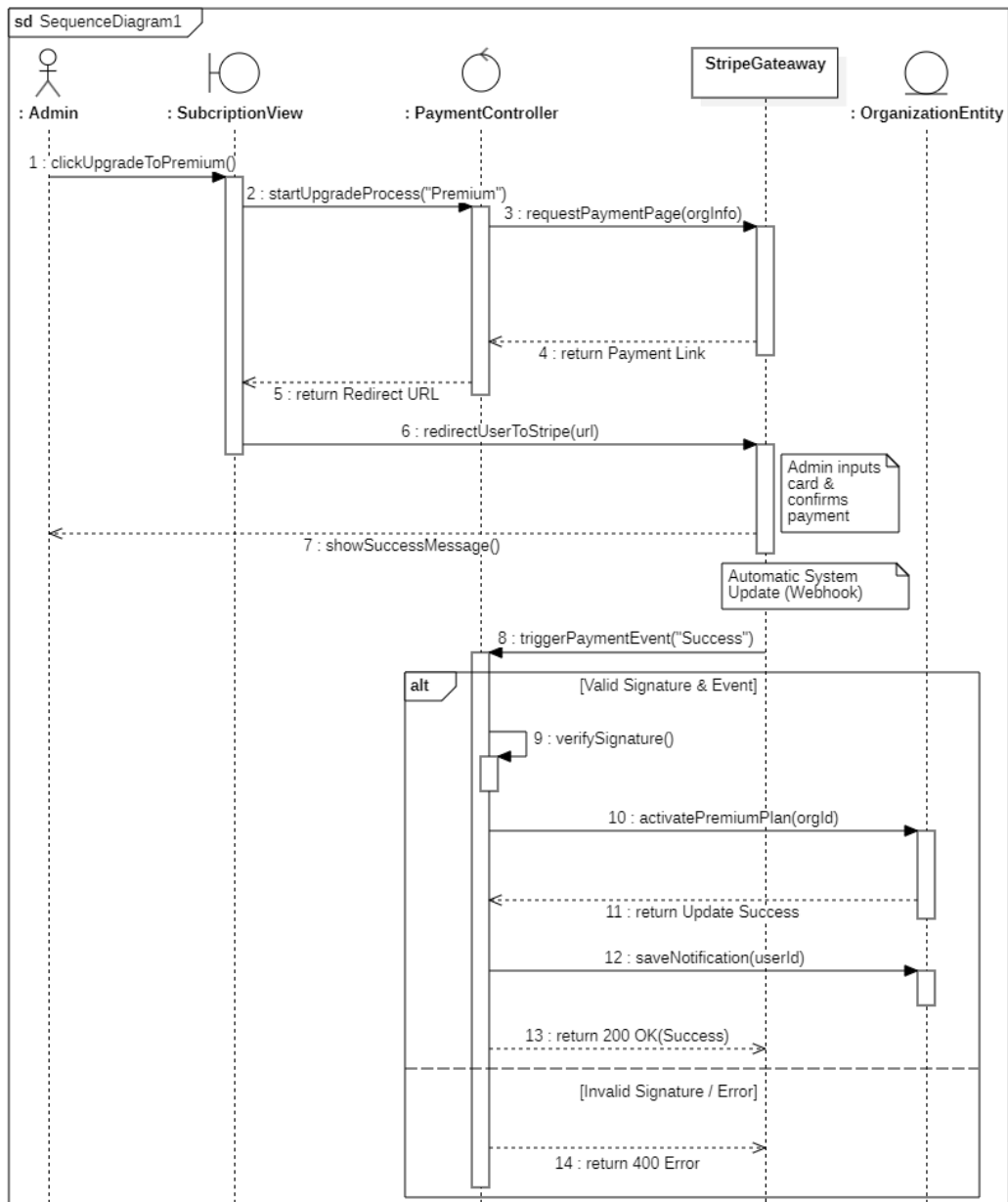


Figure 23: Premium Plan Upgrade & Stripe Payment Sequence Diagram

5.3.7. Real-time System

5.3.7.1. Real-time Chat

To ensure seamless communication, the system handles chat messages through a persistent real-time flow. As shown in the figure below, when a user sends a message, the Controller first saves it to the database to guarantee that chat history is safely retained. Immediately after saving, the system broadcasts the message to the Project Room, updating the interface for all active members instantly without requiring a page refresh.

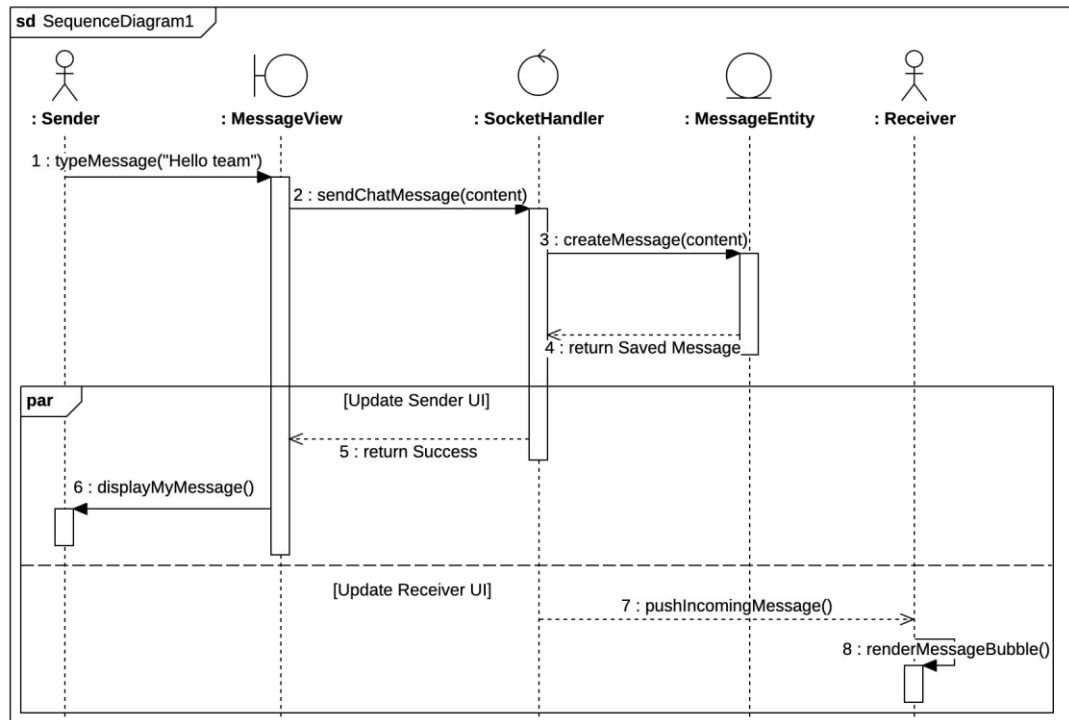


Figure 24: Real-time Chat Sequence Diagram

5.3.7.2. Notification & Mention

To manage alerts efficiently, the system supports a full lifecycle from retrieval to status updates. As shown in the figure below, when a user opens the notification center, the Controller executes parallel database queries to simultaneously fetch the latest message list and the unread count. Users can then interact with the data by marking individual items as read or using the "Mark All as Read" feature; both actions trigger immediate database updates to synchronize the notification status across all devices.

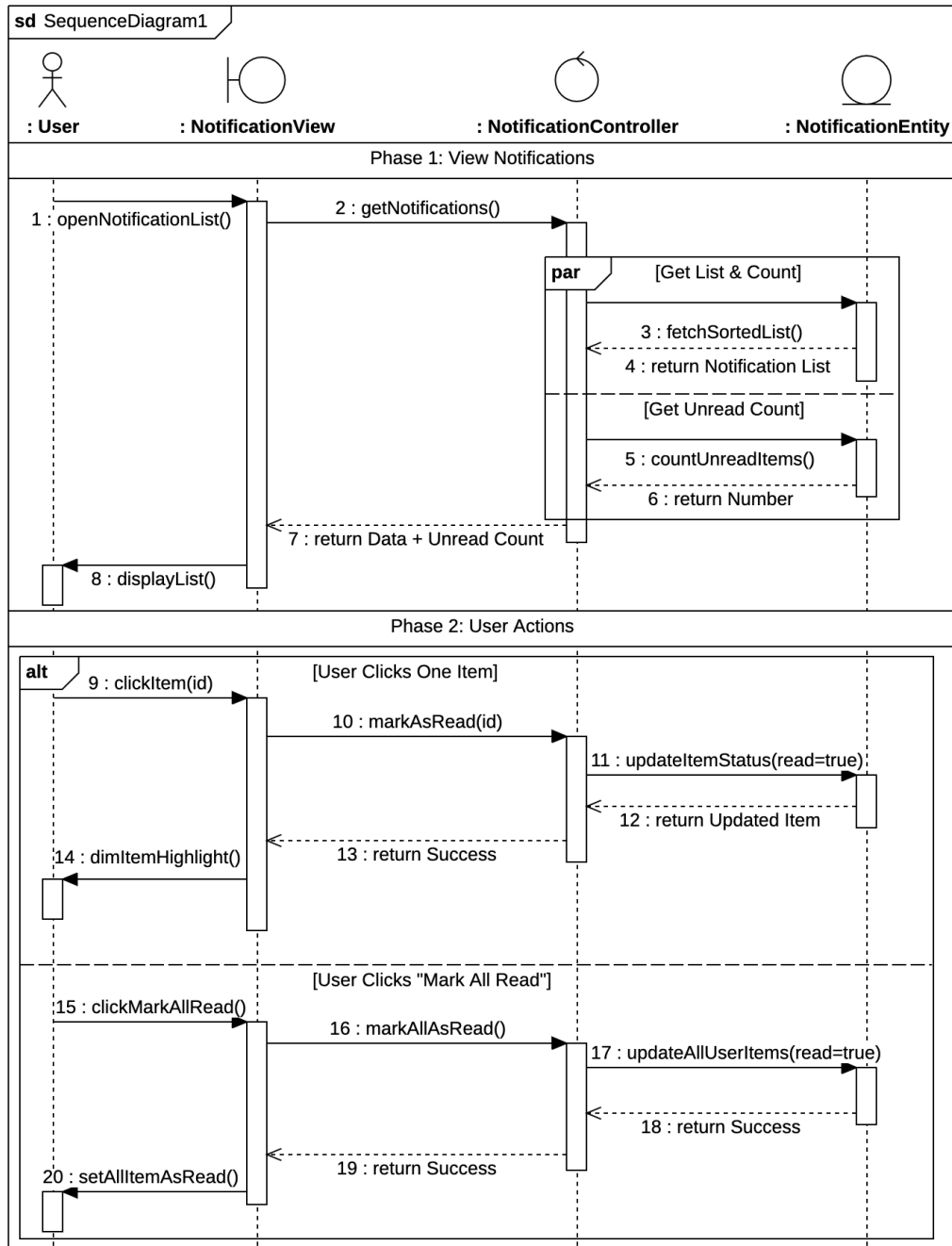


Figure 25: Notification & Mention Sequence Diagram

5.3.8. Task Management

Task Editing: Project Managers retain full authority to modify task metadata, including Title, Description, Priority, and Deadlines.

Drag & Drop Progression: The action of moving a task is strictly restricted to the Assignee. User cannot move a task assigned to another member, only drag-and-drop by themselves.

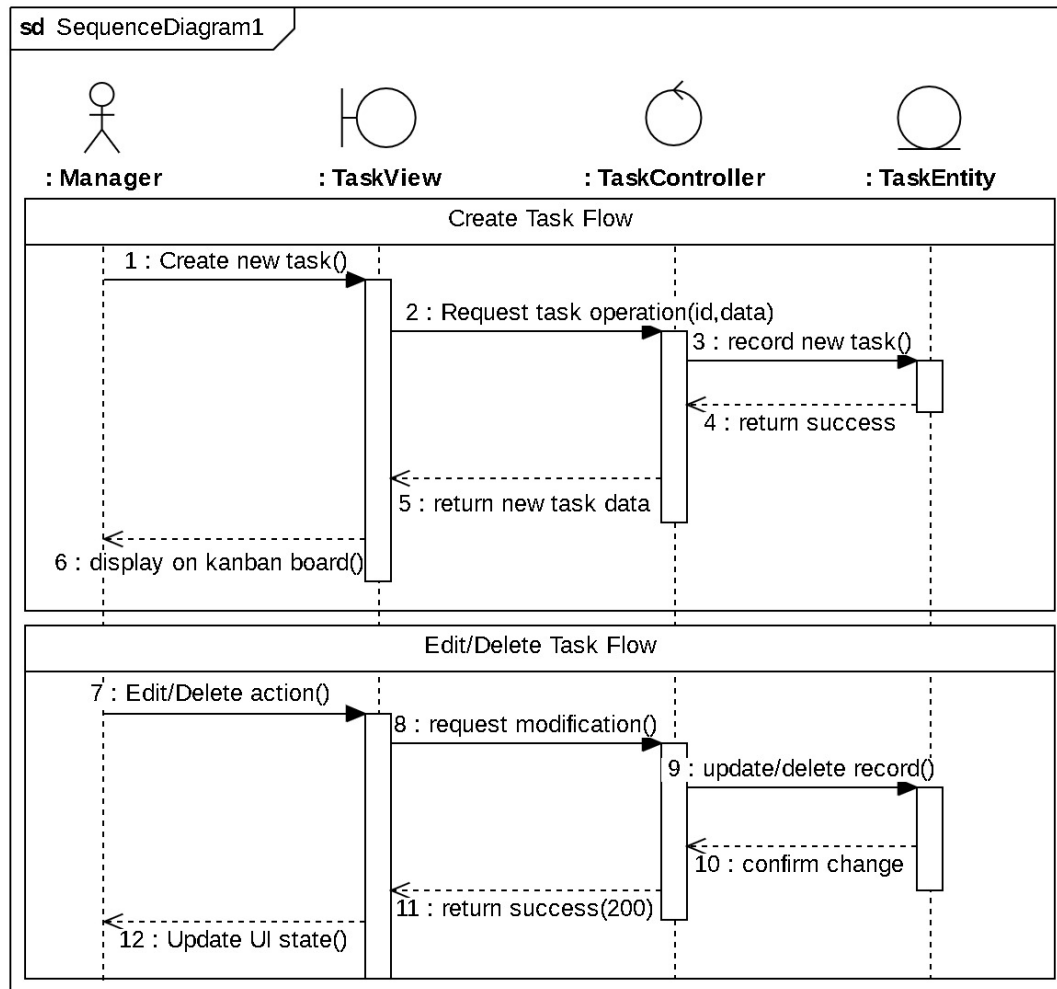


Figure 26: Create & Edit Task Sequence Diagram

5.3.9. Kanban Operations

To efficiently manage the order of tasks within Kanban columns, the system utilizes a Floating-Point Indexing. When a task is dropped between two existing tasks (Task A and Task B), its new `orderIndex` is simply calculated as the average:

$$\text{New position} = \frac{\text{Order Index}_{prev} + \text{Order Index}_{next}}{2}$$

Integrated with this logic is a strict Permission Control mechanism to ensure workflow smoothly.

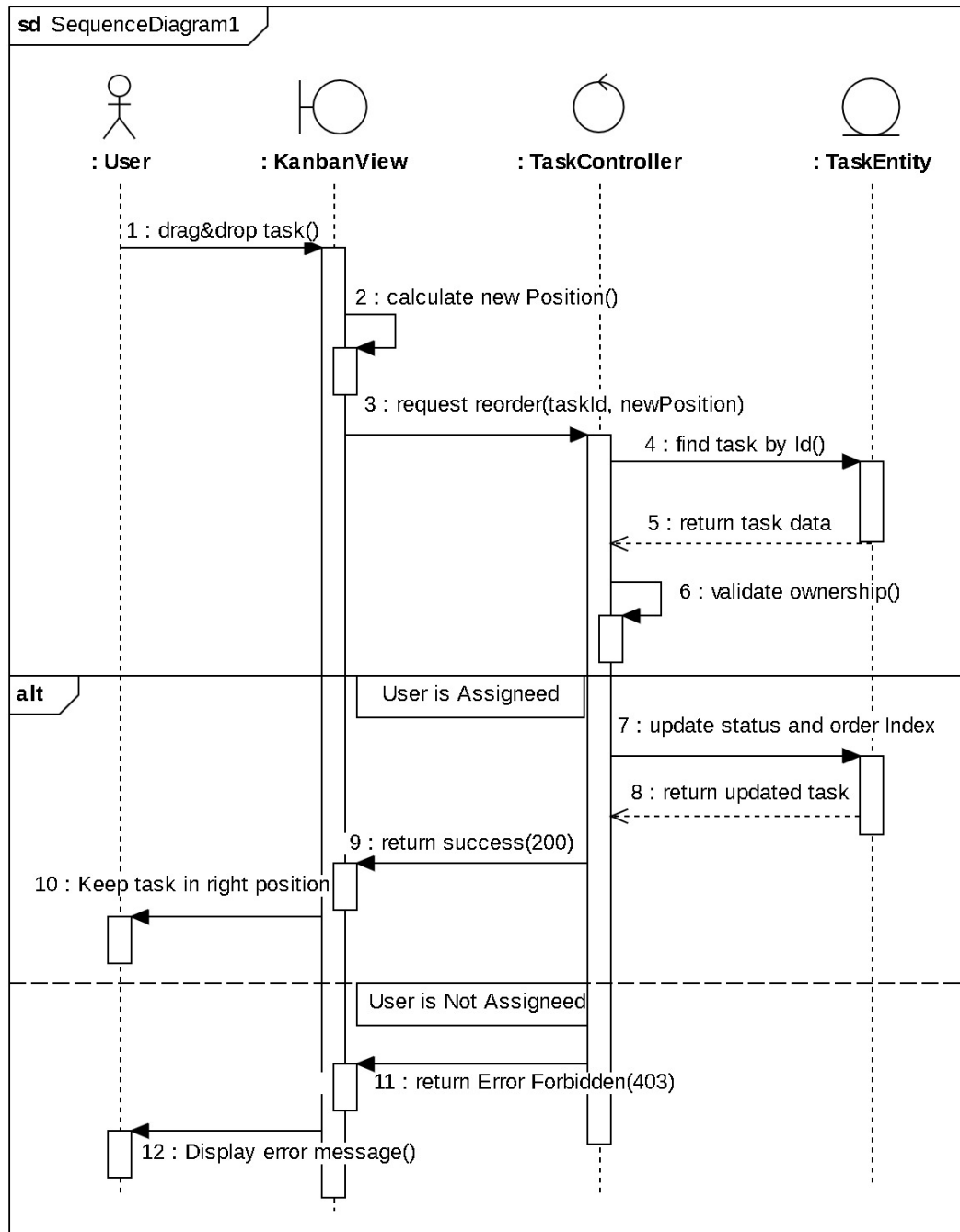


Figure 27: Kanban Task Reordering Sequence Diagram

5.3.10. Attendance Security

To ensure attendance is recorded honestly, the system uses a strict IP verification process. First, the Organization Admin sets up a 'Whitelist' of trusted office networks. Then, whenever a member clicks 'Check-in', the system automatically detects their device's public IP address and compares it against this approved list. If the IP matches, the system accepts the check-in and instantly marks the status as 'Present' or 'Late' based on the time. However, if the user attempts to check in from an unauthorized location such as a home network the request is immediately blocked with a 'Forbidden' error to prevent.

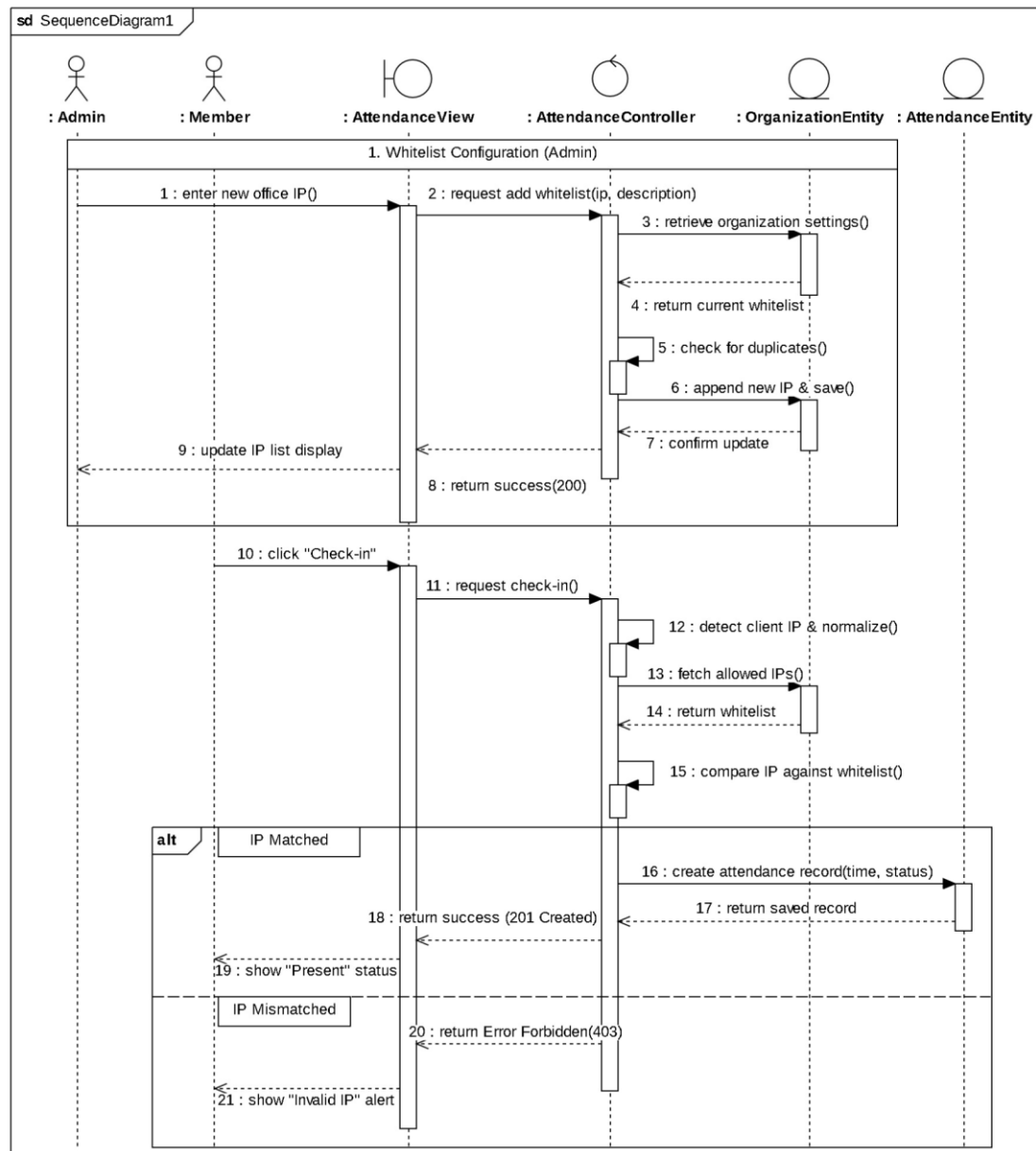


Figure 28: Attendance System Sequence Diagram

5.3.11. AI Service Integration

To boost personal productivity, the system integrates intelligent automation features. As illustrated in Diagram, the Magic Subtask workflow allows the AI to analyze task details and automatically generate a structured checklist, which is then saved directly to the database. Meanwhile, the Get Daily Brief process optimizes performance by fetching today's tasks and meeting.

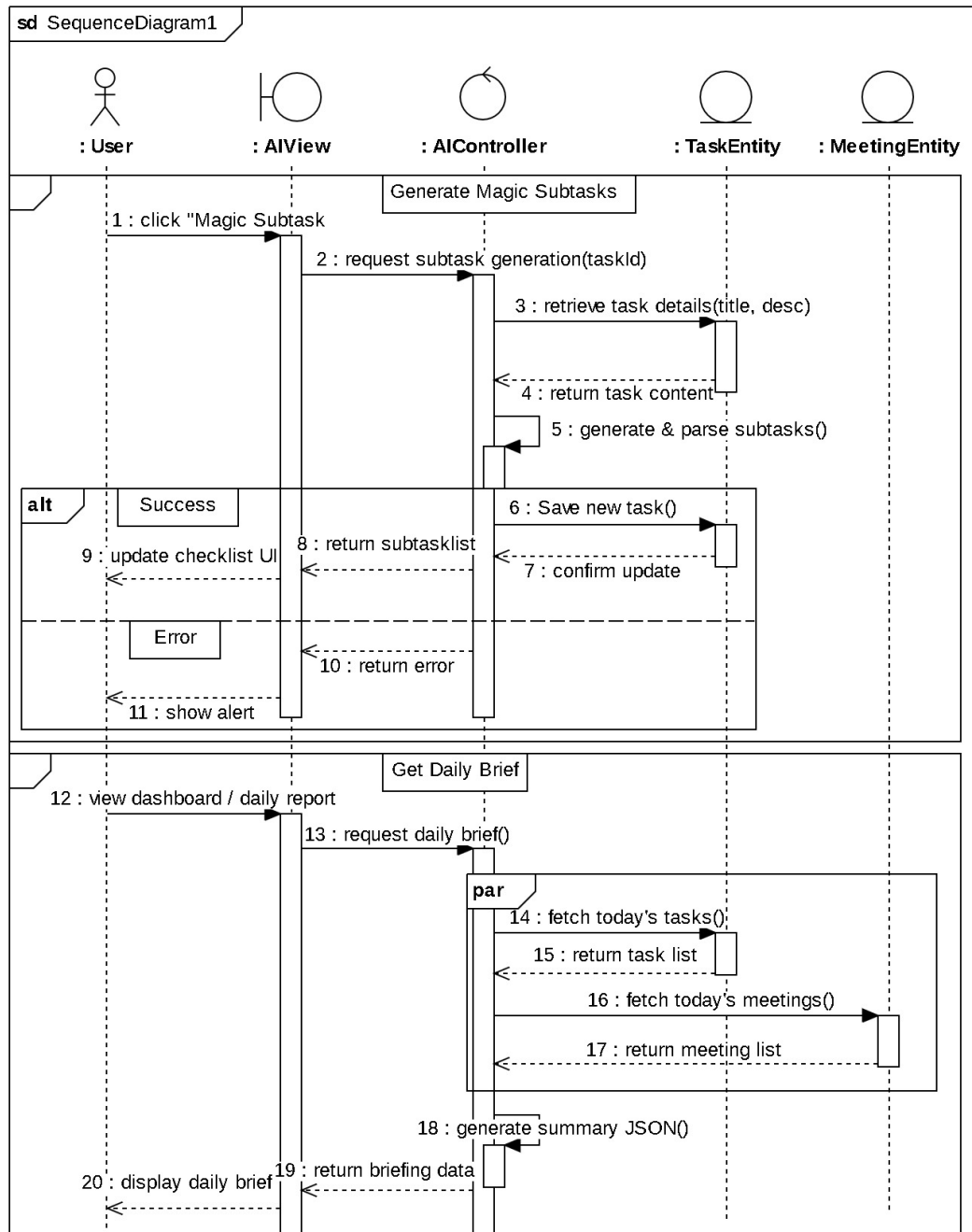


Figure 29: AI - Magic Subtask & Daily Brief Sequence Diagram

5.3.12. Meeting Management

To prevent scheduling clashes, the system automatically checks for time conflicts before creating a meeting. As shown in the figure below, when a Manager submits a schedule, the Controller verifies the time slot against existing records in the database. If the slot is busy, the system immediately blocks the request; if free, it automatically adds all active project members as attendees and sends them instant notifications, ensuring the entire team is informed without manual effort.

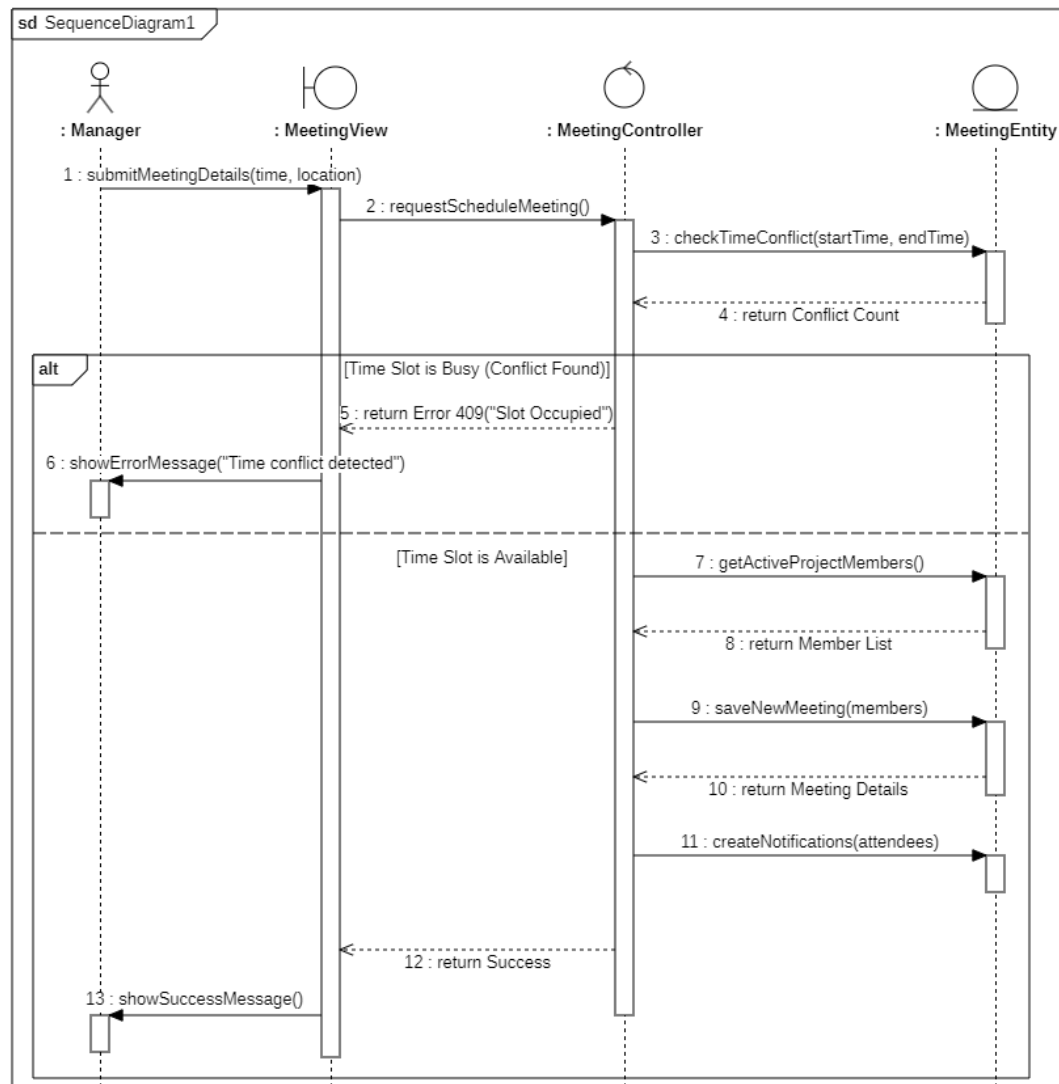


Figure 30: Meeting Scheduling Sequence Diagram

5.3.13. Project Administration

This sequence diagram describes how an Admin creates a new project within the system. This user must be an Organization Admin with the authority to manage the project lifecycle. When an Admin wants to create a project, the system requests the actor to enter the project details through the Project Management View. Once the Admin enters the details, the system will check if the organization's plan and project quota are valid and create the project. Otherwise, the system displays an error message and informs the user that they have reached the project limit for their current plan.

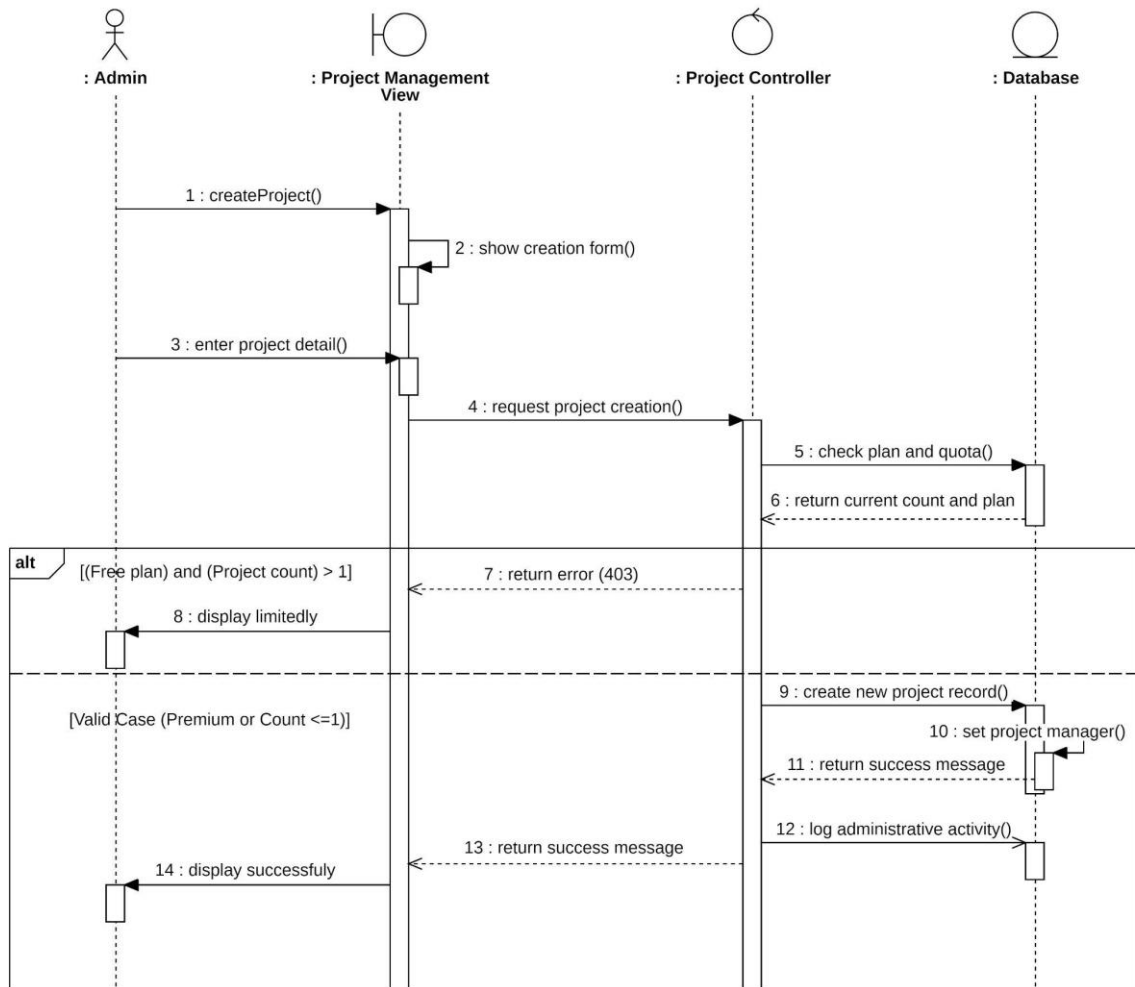


Figure 31: Project Creation & Plan Validation Sequence Diagram

5.4. Frontend Implementation

5.4.1. Technology Stack & Libraries

To support the complex requirements of a project management platform, we utilized the following stack:

1. Core Framework: ReactJS The system is built on ReactJS, utilizing its component-based architecture and Virtual DOM for efficient rendering and maintainability. React Router is employed for seamless navigation and handling dynamic routes within the Single Page Application (SPA).

2. State Management: Context API We implemented React Context API to manage global application states such as user authentication (AuthContext), project selection (ProjectContext), and real-time notifications, ensuring seamless data flow across components without prop-drilling.

3. User Interface & Styling

- **Tailwind CSS:** Used as the primary utility-first CSS framework for rapid and responsive UI development.
- **Ant Design:** Integrated for complex UI components like Statistics, Progress bars, and Spinners in the Dashboard.

- **Icons:** We utilized a combination of `@heroicons/react`, `lucide-react`, and `@ant-design/icons` to provide a rich and consistent visual language.
- 4. Data Visualization (Dashboard)**
- **Apache ECharts (echarts-for-react):** Chosen for the dashboard analytics to render complex, interactive charts (Pie charts, Bar charts) representing project status and task priorities accurately.
- 5. Task Management & Interaction**
- **Drag & Drop:** `@hello-pangea/dnd` is implemented for the Kanban board, enabling smooth drag-and-drop interactions for task status updates.
 - **User Mentions:** `react-mentions` is used in the task detail view to allow users to tag team members in comments.
- 6. Utilities & Notifications**
- **Alerts & Modals:** `SweetAlert2` is used for confirmation dialogs (e.g., deleting a task), providing a better user experience than default browser alerts.
 - **Toasts:** `react-toastify` handles temporary popup notifications for success or error messages.
 - **Celebration Effects:** `react-confetti` creates a celebratory animation upon successful payment.
 - **Date Formatting:** `date-fns` is employed to manipulate and format dates (e.g., "2 hours ago") efficiently.
- 7. Communication & Authentication**
- **Axios:** Handles all HTTP requests to the backend with interceptors for token management.
 - **Socket.io-client:** Establishes a WebSocket connection for real-time features like chat and instant notifications.
 - **Google OAuth:** `@react-oauth/google` facilitates secure and quick login via Google accounts.

5.4.2. Key Screens & UI Components

5.4.2.1 Authentication (Login/Register)

Purpose: Acts as the secure gateway for the application. It handles context-aware routing, distinguishing between returning users and new sign-ups

Components:

- **Visual Layout:** The interface is constructed using a centered card layout (div with fixed width and padding) overlaid on a full-screen background image. It employs strict CSS backdrop-filter properties via Tailwind classes (`backdrop-blur-[30px]`, `bg-white/20`, `border-white/30`) to achieve a modern "glassmorphism" aesthetic, ensuring visual hierarchy and readability.
- **Interactive Forms:** Built using React controlled components for state management. Visual cues are enhanced with `lucid-react` icons.
- **GoogleLogin:** Integrated from `@react-oauth/google` to facilitate third-party authentication.
- **Navigation Logic:** The `handlePostSignupRedirect` function utilizes `react-router-dom` hooks (`useNavigate`, `useLocation`) to inspect the state route users to the Pricing Page or the Pending Page.

5.4.2.2 Dashboard (Home)

Purpose: Serves as the central command center, aggregating data from the DashboardService and rendering role-specific analytics and AI insights in a unified view. It provides a snapshot of the organization's health and personal productivity.

Components:

- **Dynamic Header:** The top navigation bar includes dropdowns for "Project", "Month", and "Year" filtering.
- **Role-Based Rendering:** The main layout utilizes conditional rendering logic (Admin's View, Manager's View, Member's View). The system checks the user's role and dynamically injects the corresponding UI blocks.
- **Task Summary:** A custom sub-component that renders a horizontal list of metric cards. It receives a summaryDataprop and maps it to individual cards displaying icons and numerical values for Total, Todo, In Progress, Done, and Overdue tasks.
- **Data Visualization:**
 - **ReactECharts:** ReactECharts to render interactive analytics. Use helper functions like getAdminBarOption to generate config objects for "Task Priority" and "Daily Activity."
 - **Stats:** Use Ant Design components (Card, Statistic, Progress) organized in a responsive grid to display high-level numbers.
- **Activity Feed:** A custom list section that renders recent activities fetched from the backend.
- **AI Daily Widget:** Develop an isolated, floating component that fetches AI insights independently.

5.4.2.3. Project Workspace (Kanban Board)

- **Drag-and-Drop Core:** Built on @hello-pangea/dnd. We utilize DragDropContext to wrap the board, enabling complex coordinate calculations for draggable task cards.
- **Smart Filtering:** Implements instant client-side filtering. A useEffect hook triggers JavaScript array methods to refine the visible task list immediately when users select labels or assignees.
- **Rendering:** Dynamically maps task objects to Tailwind CSS components, handling conditional logic for Priority badges and Status columns.

5.4.2.4. Task Details

- **Data Architecture:** Uses react-router-dom for routing and Promise.all to fetch task metadata and comments in parallel, ensuring a fast load time.
- **AI Subtask:** The "AI Suggest" feature incorporates a setTimeout cooldown mechanism to programmatically lock the button and prevent API abuse.
- **Comments:** Integrates react-mentions to handle user tagging. The input detects the @ trigger and formats tags using specific markup patterns for storage.
- **Edit Mode:** Utilizes conditional rendering to seamlessly swap between static text displays and editable inputs (<textarea>, <select>) based on user interaction.

5.4.2.5. Calendar

Purpose: The Calendar page serves as the central hub for time management and personnel tracking within the application. It allows members to log their daily presence while providing administrators with high-level reporting tools and security configurations.

Components: The page uses a split layout design divided into two primary interactive areas:

- **Split Layout Architecture:** Divided into 2 parts, a CalendarPanel for the monthly view and a RightPanel that toggles between Member actions (Check-in) and Admin tools.
- **Create Meeting:** A form interface for scheduling new meetings which include Project selection, Title, Start/End time, Location, and Description.
- **Meeting Detail:** Displays full details of a selected event. Includes conditional rendering for the Delete action (available only to the creator or Admins).
- **Attendance Report(Admin feature):** A data table summarizing monthly attendance consists of Total Check-ins, On-Time records, Late days, and Absences. And a sub-component Report Detail that shows the exact day and time a member was late or absent.
- **IP Management:** A management interface to Whitelist IP addresses for valid attendance check-ins.

5.4.2.6. Member

Purpose: The page is designed as the visual interface for personnel administration, allowing administrators to scan member statuses and perform management actions efficiently.

Components:

- **Search Toolbar:** Features a search input with a MagnifyingGlassIcon absolute-positioned inside, allowing users to search a member by the first letter of their name.
- **Invite Button(Admin/Manager feature):** It fetches a unique project code, offers a one-click "Copy to Clipboard" feature, and allows generating a new code if the old one is compromised.
- **Role Selection (Admin feature):** allowing Admins to update member permissions via dropdowns.

5.4.2.7. Projects (Admin only)

Purpose: It is designed to provide a high-level overview of all ongoing projects while allowing users to drill down into specific analytics and team management.

Components:

- **View Switcher:** A toggle component allows users to switch between a Grid View and a List View.
- **Project Card:** It composes reusable sub-components including color-coded StatusBadge, dynamic ProgressBar, and metadata icons.
- **NewProjectModal:** Visualizing the form, which includes project name, description, manager and deadline, to create a new project.
- **ProjectDetailDrawer (Detailed Analytics):** We use ReactECharts for task distribution and custom animated SVG circular progress bars, while the team section displays members in card format with color-coded role badges and a floating action button that shows project chat.

5.4.2.8. Notifications

Purpose: The Notification Page acts as a central hub for all user alerts and updates. Its primary role is to inform system events (mentions, task assignments, new members, meeting events) into a unified, interactive list.

Components:

- **Filter Toolbar:** Built with toggle controls for switching between "All" and "Unread" views, featuring a "Mark all read" action that updates local state and backend status.

- **Notification Item:** Each notification card is a smart component that adapts its visual presentation based on the type prop. New notifications are highlighted with a subtle blue background and a solid indicator dot.
- **NotificationContext:** Establishes a persistent WebSocket connection to ensure the feed and unread counters are updated instantly without page reloads.

5.4.2.9. Main Layout Components

Side Bar

Purpose: Establishes the global navigation structure and persistent utility context (Chat, Alerts, User Menu) across the application.

Components:

- **Expandable Container:** The sidebar utilizes a "Hover-Intent" design pattern to maximize screen real estate, defaulting to a compact rail that smoothly expands to a full width of 64 pixels upon user interaction.
- **Sidebar Item:** Navigation links are designed with adaptive visual feedback, utilizing `useLocation` to highlight the active route with the brand color.
- **ProjectSwitcher:** Located at the bottom as a global scope controller. It uses lazy state initialization to render an animated dropdown for workspace toggling without UI flickering.

Navbar

Purpose: Its primary function is to establish immediate context via page titles. It also acts as the central hub for communication (Chat), alerts (Notifications, Join Requests), and user account management.

Components:

- **Upgrade Plan Button:** This button for free admin plan to upgrade easier.
- **Join Request Bell(Admin/Manager only):** Displays a real-time numeric badge that triggers a sliding side-drawer for immediate member approval/rejection.
- **Notification Bell:** The Notification Bell employs a click-triggered dropdown menu to present color-coded updates and utilizes smart navigation logic to automatically route users to specific contexts.
- **User Profile:** Containing a Profile link and Logout button.
- **Chat box:** Functions as a communication overlay, utilizing React Portals to render the widget directly into the document body. The internal conversation view implements smart message grouping logic that only renders avatars when the sender changes. The interface ensures usability through an auto-scroll behavior triggered by new socket events and includes a polished input footer with a rounded text field and a rotated paper airplane icon for submission.

5.4.3. UI/UX & Interactive Logic

Our frontend strategy focuses on three core pillars: adaptability, perceived speed, and user safety.

- **Responsive Adaptability:** Utilizes Tailwind CSS to create a fluid layout where complex elements, specifically the Kanban grid, adapt from a vertical stack on mobile to a multi-column view on desktop.
- **Optimistic UI:** Implements instant state updates for critical interactions (like dragging tasks or checking in) to mask network latency while the server synchronizes in the background.

- **Interactive Safeguards:** Features interaction locking via loading spinners during AI generation to prevent duplicate requests. Destructive actions are secured using SweetAlert2 confirmation modals.
- **Real-time Feedback:** Integrates react-toastify for non-intrusive status alerts. Socket events are employed to push live updates, such as task movements, instantly to all clients without page reloads

5.5. API Documentation

These are all the endpoints for the API we have been building.

Module: Authentication

Method	Endpoint	Functional Description
POST	/auth/signup	Register a new account.
POST	/auth/login	Login to the system.
GET	/auth/me	Get the current user profile.
PATCH	/auth/profile	Update profile details.
POST	/auth/change-password	Change login password.
POST	/auth/switch-org	Switch active workspace.

Table 17: Authentication API Endpoints

Module: Projects & Collaboration

GET	/projects	List all accessible projects.
POST	/projects	Create project (checks Free Plan limit).
GET	/projects/:id	Get project details.
PUT	/projects/:id	Update project settings.
DELETE	/projects/:id	Soft delete project.
POST	/projects/join	Join project via invite code.
GET	/projects/:id/members	List project members.
POST	/projects/:id/members	Add members manually.
GET	/projects/:id/summary	Get dashboard statistics.
GET	/projects/:id/invite-code	Get a join invite code.

Table 18: Projects & Collaboration API Endpoints

Module: Task Management

GET	/projects/:id/tasks	List all tasks in a project.
POST	/projects/:id/tasks	Create new task.
GET	/tasks/:id	Get task details.
PUT	/tasks/:id	Update task info.
PATCH	/tasks/:id	Update task status.
PATCH	/tasks/reorder	Update task position on Kanban board.
DELETE	/tasks/:id	Soft delete task.
POST	/tasks/:taskId/subtasks	Add manual subtask.
POST	/tasks/:taskId/magic-subtasks	Auto-generate subtasks using AI from description.
POST	/tasks/:taskId/attachments	Add file attachment to task.

Table 19: Task Management API Endpoints

Module: Notification & Alerts

GET	/notifications	List notifications & unread count.
PATCH	/notifications/:id/read	Mark single notification as read.
PATCH	/notifications/read-all	Mark all notifications as read.

Table 20: Notification & Alerts API Endpoints

Module: Dashboard & Analysis

GET	/dashboard/admin-stats	Get organization-wide statistics.
GET	/dashboard/manager-stats	Get statistics for managed projects.
GET	/dashboard/member-stats	Get personal performance & activity.

Table 21: Dashboard Analysis API Endpoints

Module: Payment & Subscription

POST	/payment/session	Create Premium checkout link.
POST	/payment/webhook	Sync subscription status.
POST	/payment/cancel	Schedule cancellation at period end.
POST	/payment/resume	Reactivate cancelled subscription.
POST	/payment/portal	Open billing management portal.

*Table 22: Payment & Subscription API Endpoints***Module: Meeting & Scheduling**

Method	Endpoint	Functional Description
GET	/projects/:projectId/meetings	List all project meetings.
POST	/projects/:projectId/meetings	Schedule meeting (auto-checks conflicts & invites members).
GET	/meetings/:id	Get meeting details.
PATCH	/meetings/:id	Update meeting details.
DELETE	/meetings/:id	Cancel meeting (soft delete).

*Table 23: Meeting & Scheduling API Endpoints***Module: Real-time Chat**

Method	Endpoint	Functional Description
GET	/projects/:projectId/messages	Fetch project chat history.

*Table 24: Real-time Chat API Endpoints***Module: Organization**

Method	Endpoint	Functional Description
POST	/organizations	Create a new organization & assign admin role.
GET	/organizations/current	Get current organization details.

*Table 25: Organization API Endpoints***Module: Attendance**

Method	Endpoint	Functional Description
POST	/attendance/check in	Submit daily check-in.
GET	/attendance/me/today	Check personal status for today (Present/Absent).
GET	/attendance/me	Retrieve personal attendance history.
GET	/attendance/all	Get organization-wide attendance report (Admin/Manager only).
GET	/attendance/my-ip	View current IP address for debugging.
GET	/attendance/whitelist	List all whitelisted IPs for the organization.
POST	/attendance/whitelist-ip	Add a new trusted IP to the whitelist.
PUT	/attendance/whitelist-ip/:ipId	Update an existing IP configuration.
DELETE	/attendance/whitelist-ip/:ipId	Remove an IP from the whitelist.

*Table 26: Attendance API Endpoints***Module: Comments & Collaboration**

Method	Endpoint	Functional Description
GET	/tasks/:taskId/comments	Retrieve all comments for a specific task.
POST	/tasks/:taskId/comments	Post a comment & trigger @mention notifications.

Table 27: Comments & Collaboration API Endpoints

6. RESULTS AND DISCUSSION

6.1. Functionality Tests

Feature	Test Case	Expected Result (Guideline)	Status
1. Auth & Onboarding	Registration	- User can fill out and submit the registration form. - A new user record is successfully created in the system database. - Error messages appear for invalid or incomplete input.	PASS
	Login	- User can log in with valid email and password. - Incorrect credentials trigger an error message. - System generates a valid JWT upon success.	PASS
	Context management	- The system correctly identifies the currentOrganizationId. - Users can switch between different organization contexts. - All workspace data updates to match the selected context.	PASS
2. Member Management	Add member / Join Request	- Users can enter a valid 6-digit invite code. - The system validates the code and sends a join request to the project manager.	PASS
	Join Approval	- The manager can view the list of pending join requests. - Clicking "Approve" successfully adds the user to the ProjectMember collection. - The approved user gains access to the project workspace immediately.	PASS
	Member search	- Users can search for members by name or email within the project. - Results are filtered accurately based on the search query. - Search results display correct member profiles and roles.	PASS
	Role Update	- Admin can select a member and choose a new role (Manager or Member). - The database record is updated with the	PASS

		new role assignment. - The UI updates to reflect the new role and its associated permissions.	
	Remove member	- Admin can initiate the removal of a member from the project. - The system performs a "soft-delete" on the member's account access within that project. - The removed user is logged out or redirected from the project view.	PASS
3. Project Workspace	Create New Project	- Admin can enter required project details (Name, Deadline) and initiate creation. - System verifies the organization's subscription plan and existing project count. - For a valid request, a new project is created. - If the project limit for a Free plan is exceeded, the system blocks the action and displays a "403 Forbidden" error.	PASS
	Invite Code Access	- Users can enter a unique 6-digit invite code provided by the project manager. - The system validates the code against the project database to ensure it is active and correct. - Upon successful validation, the user is granted project access and automatically redirected to the project workspace.	PASS
4. Task Management & Security	Create Task	- Users can enter task title, description, and assign a member. - Task appears in the "Todo" column instantly without needing to refresh the page - Data is correctly stored in the database with the proper projectId for isolation.	PASS
	View Task Details	- Users can click on a task card to view full information. - All details, including description, assignees, and labels, load correctly in the modal.	PASS
	Delete Task	- Authorized user (Manager) selects the delete option for a specific task. - Task is removed from the Kanban board and the database immediately	PASS
	Drag & Drop	- Users can drag a task to a different column or change its position. - Task order is saved using smart indexing,	PASS

		and the change is synced in real-time for other members.	
	Add Labels	- Users can create or select customizable, project-scoped labels for a task. - Labels are displayed clearly on the task card with their assigned colors.	PASS
	Task Attachments	- Users can add external resource links or file attachments to the task context. - Links are clickable and successfully direct the user to the attached resource.	PASS
	Edit Assigned Task (Member)	- A member can update the status or details of a task specifically assigned to them. - The system validates the assignment and successfully saves the changes.	PASS
	Edit Others' Task (Member)	- A member can not modify a task that is not assigned to them. - The system blocks the request and displays an "Unauthorized" or "403 Forbidden" error message.	PASS
5. Collaboration	Real-time Chat	- The system establishes a box-chat dedicated to the specific project. - Messages sent are delivered instantly to all online project members without page refresh. - Chat history loads correctly with accurate timestamps and sender identification	PASS
	Mention System (@)	- Typing the "@" prefix triggers a dropdown list of available project members. - Selecting a member successfully tags them within the comment or chat message - The mentioned user receives a real-time push notification or direct alert immediately.	PASS
	Meeting Schedule	- Users can create a meeting event and invite specific project members. - Scheduled meetings appear accurately in the project's schedule or calendar view. - The system performs a conflict check to detect overlapping time slots and returns an error if the user is busy.	PASS
6. Dashboard and	Admin View	- Dashboard displays comprehensive organization KPIs including total users and active projects.	PASS

Analytics		- Project counts are calculated and displayed correctly. - System accurately aggregates data from all projects within the specific organization context	
	Manager View	- Dashboard visualizes real-time project health and task completion rates for assigned projects. - Member activity logs and workload distribution charts are updated based on recent Kanban actions. - Managers can effectively track project progress against set deadlines.	PASS
	Member View	- Personal dashboard displays a summary of assigned pending tasks and upcoming deadlines. - Data only show information relevant to the projects the member is currently joined in.	PASS
7. Attendance	Whitelist Configuration	- Admin can access the IP configuration interface. - Valid IPv4 or Wildcard IPs (e.g., 192.168.1.*) are successfully added to the organization's whitelist. - Invalid IP formats are rejected with an error message.	PASS
	IP Detection & Check-in	- When a user clicks "Check-in", the system correctly identifies the client's public IP address. - If the IP matches the whitelist, the system records the attendance as "PRESENT" or "LATE". - The UI updates the successful check-in status.	FAIL(IP detection mismatch on proxy)
	Security Enforcement	- Users attempting to check in from a non-whitelisted network (e.g., home WiFi) are strictly blocked. - The system returns a 403 Forbidden error and prevents database insertion.	PASS
	History Reports	- Admin can query attendance logs by date range or specific project. - The system accurately aggregates data (Present/Late/Absent counts) for each member.	PASS

8. AI Services	Magic Subtask	- The user triggers the AI action for a complex task with a specific title and description. - AI successfully analyzes the task context and returns a checklist of actionable sub-steps. - The UI updates automatically to display the generated subtasks within the task modal.	PASS
	Daily Smart Summary	- The system scans the user's database for tasks due today and scheduled meetings. - AI generates a personalized work briefing including a professional greeting and key priorities. - The summary is displayed correctly on the user's dashboard, providing a clear overview of the day's highlights.	PASS
9. Payment	Plan Upgrade	- Users can initiate the upgrade process from the organization's billing management interface. - The system successfully redirects the user to a secure Stripe payment gateway for transaction processing. - Upon successful payment, the organization's plan tier is automatically updated from "Free" to "Premium" in the database. - The system immediately lifts restricted project quotas and unlocks advanced administrative capabilities.	PASS

Table 28: Functionality Tests

6.2. Performance Tests

To evaluate system stability and scalability, we conducted stress testing by simulating 100 concurrent virtual users generating over 13,000 requests. The results, summarized in the table below, demonstrate that the system maintains a 0.00% error rate under peak load, effectively handling read/write-heavy operations with acceptable response times.

Test Scenario	Key Metrics (Data)	Performance Assessment
System Stability	- Error Rate: 0.00% (Across 7,000 - 13,000+ requests)	-The system demonstrated stability. Zero failures (500/Timeout) occurred during peak load (00 concurrent users).
Read-Heavy Load (Project Dashboard)	- Load: 100 virtual users -Average Wait Time: 2.7s	-Users have to wait nearly 3 seconds to see the dashboard. This is a bit slow due to heavy database calculations.
Write-Heavy Load (Task CRUD Flow)	- Load: 100 virtual users -Average Wait Time: 1.6 seconds	-Creating and updating tasks involves complex logic, so 1.6 seconds is a safe speed to ensure data accuracy.
Processing Speed	-Speed: 26 Requests per second	-The server processes requests continuously at a steady pace without getting stuck or overloaded.
Processing Capacity	-Tested Limit: 100 Concurrent Users	-The system handled 100 simultaneous users without crashing, though response time increased linearly.

Table 29: System Performance & Load Test Summary

6.3. Visual Result

6.3.1. Pages in ADMIN role

6.3.1.1. Dashboard

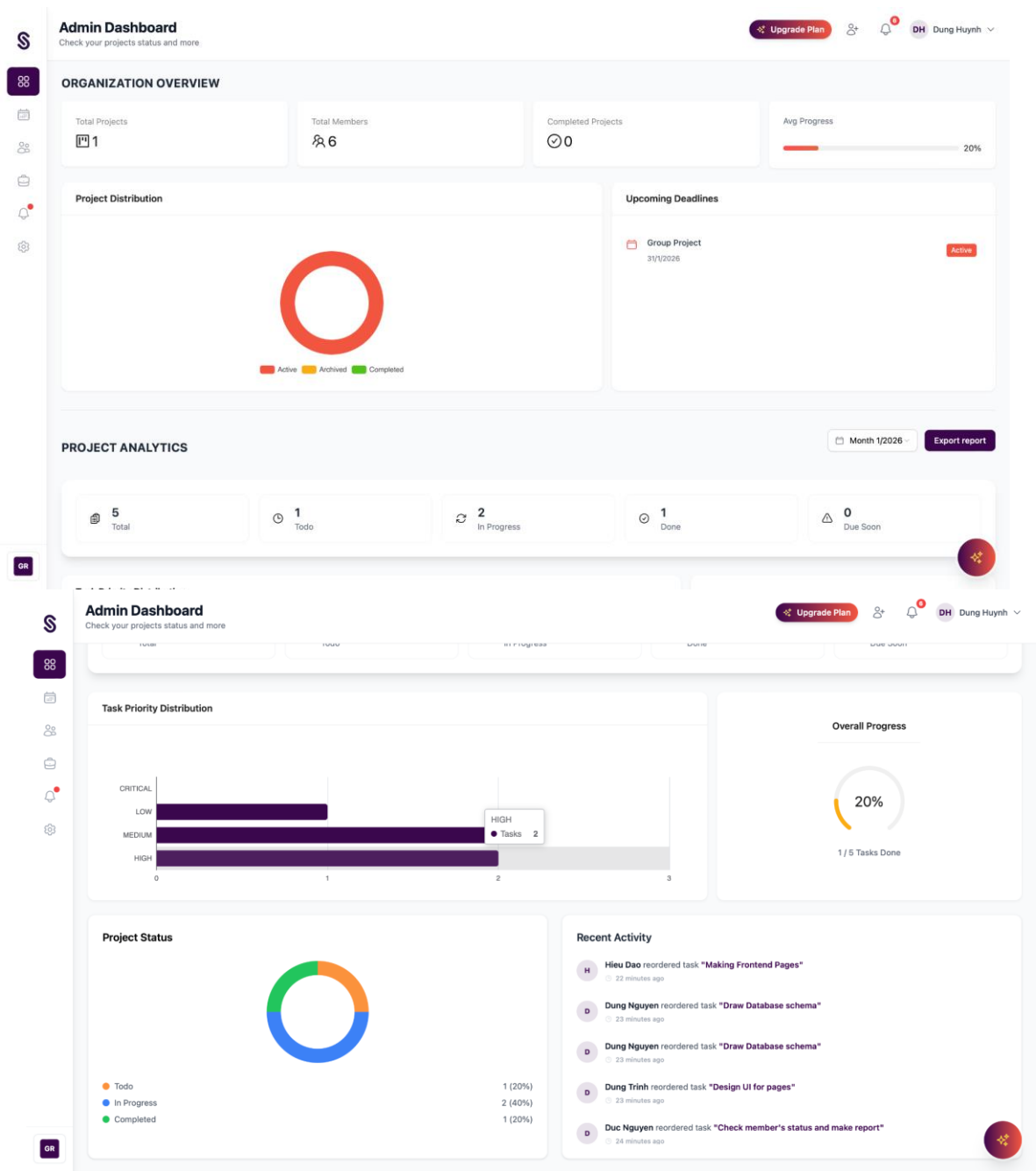


Figure 31 & 32: Dashboard layout for Admin

6.3.1.2. Calendar

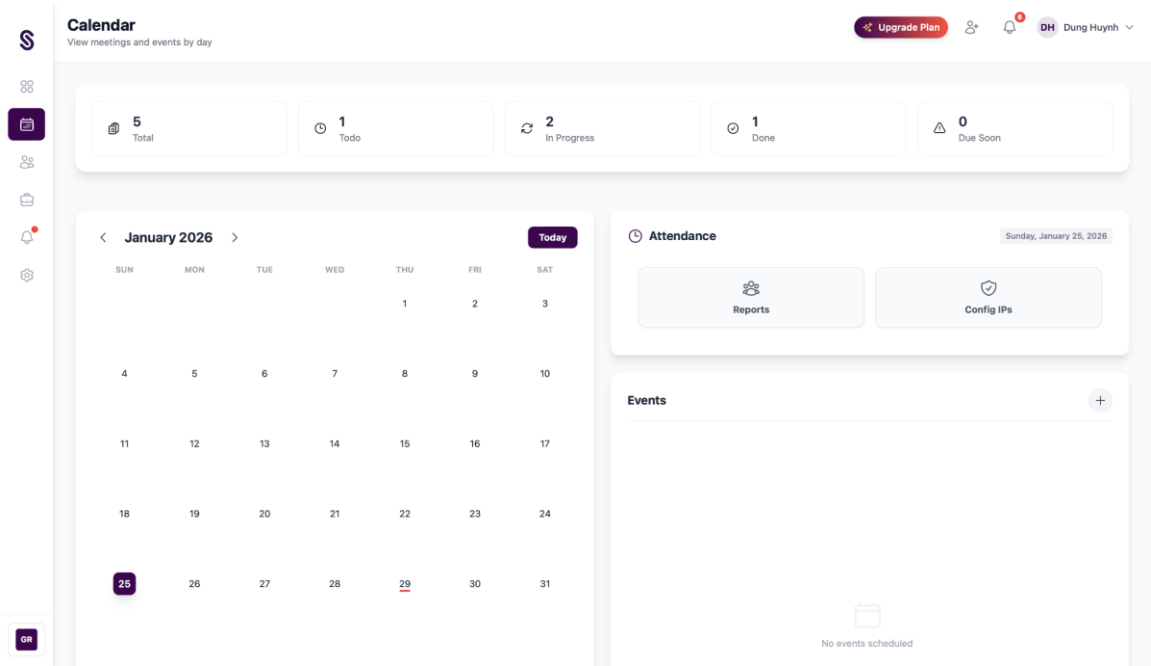


Figure 33: Calendar layout for Admin

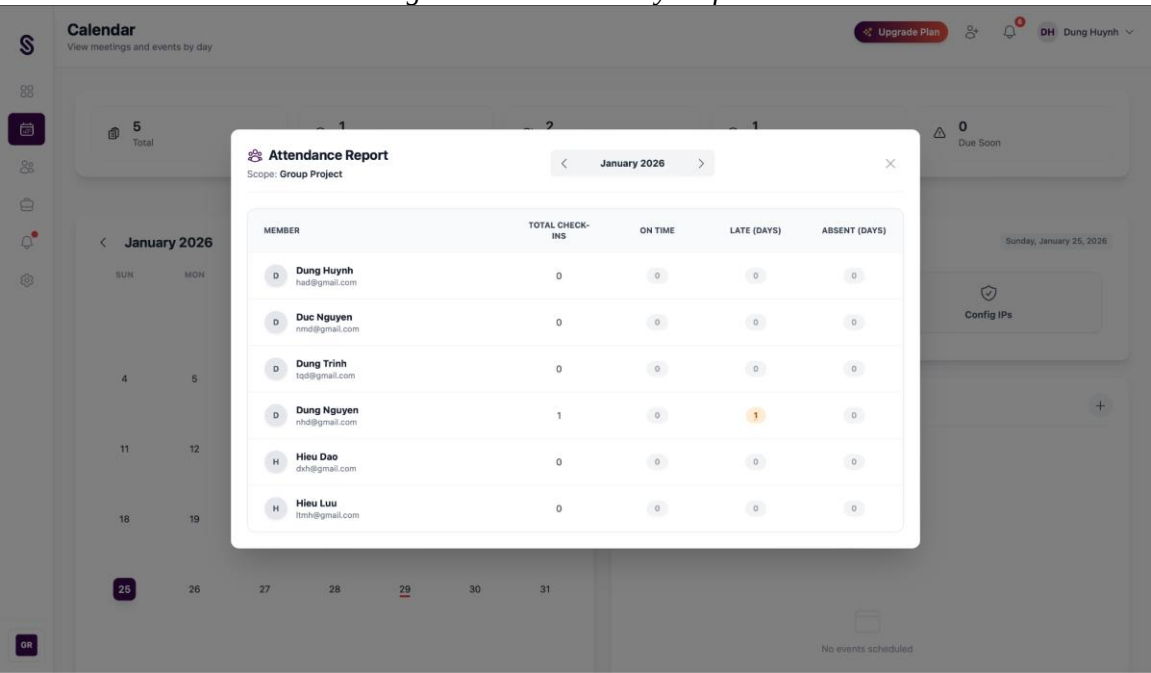


Figure 34: Attendance Report

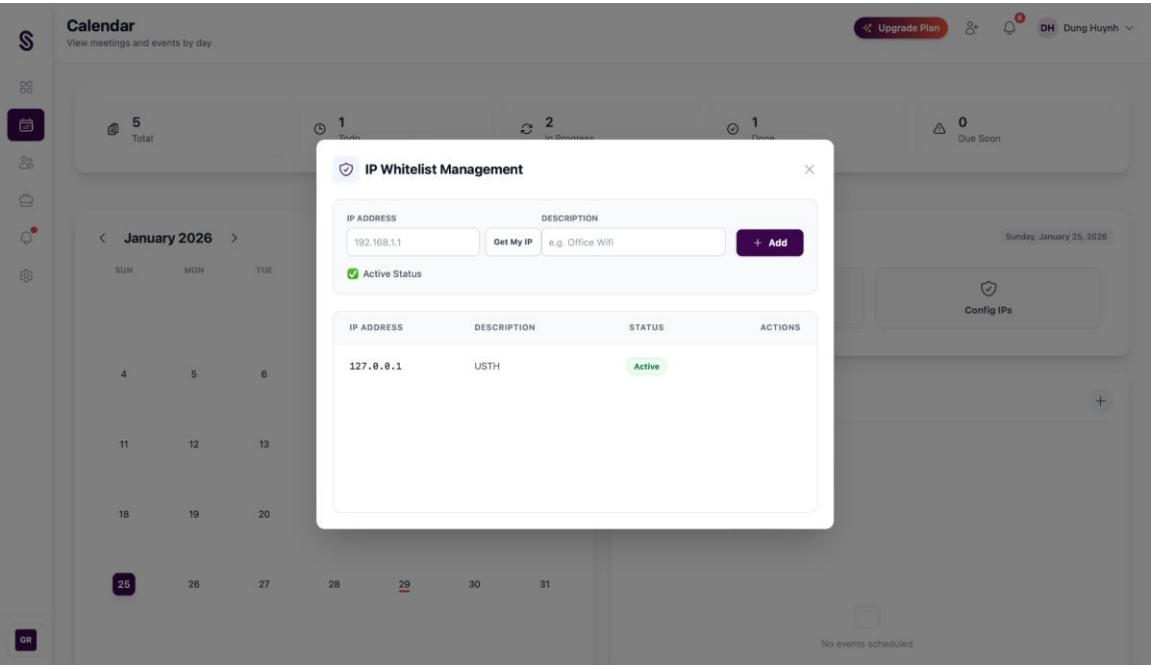


Figure 35: Config IPs to check attendance

6.3.1.2. Members

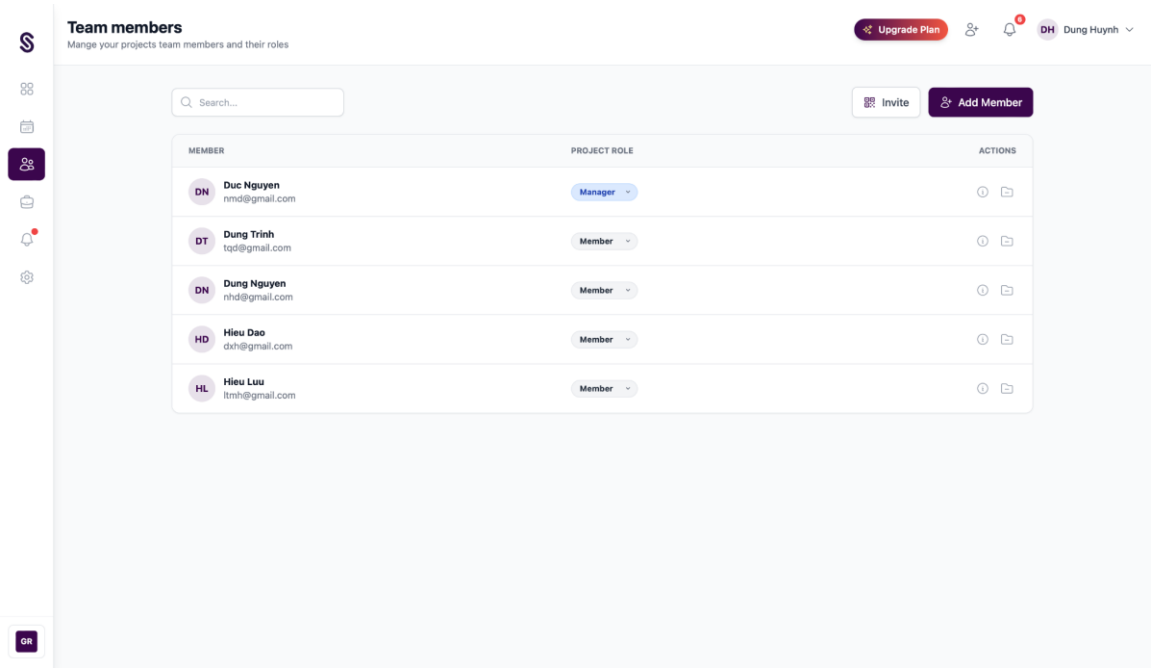


Figure 36: Members layout for Admin

6.3.1.3. Projects

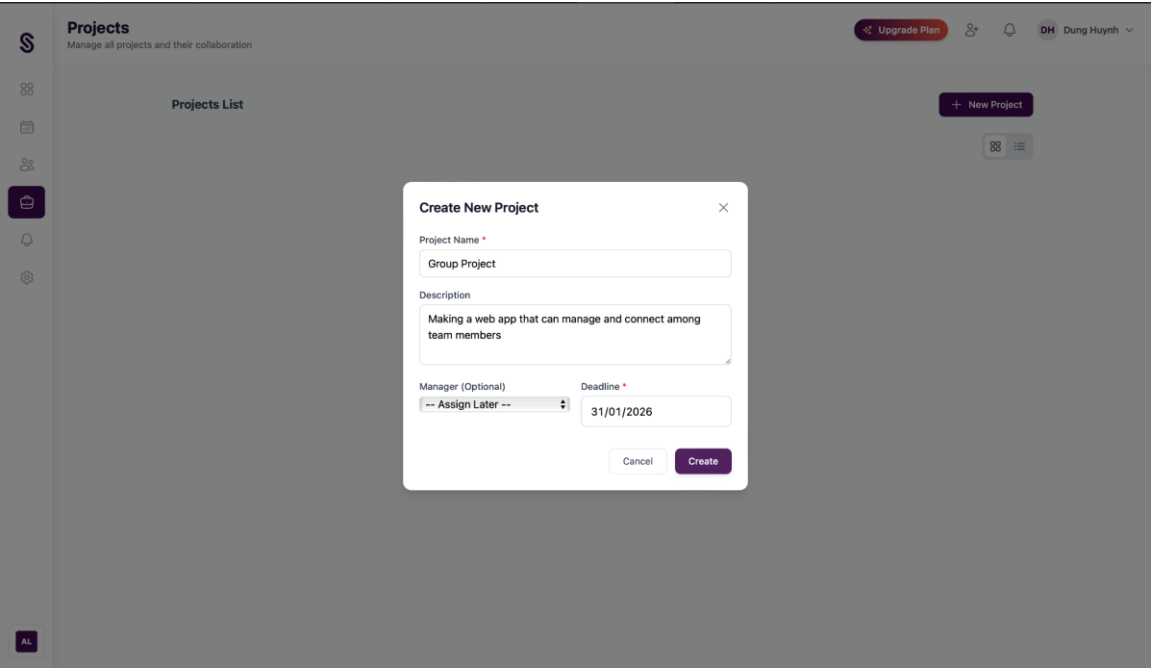


Figure 38: Create Project form

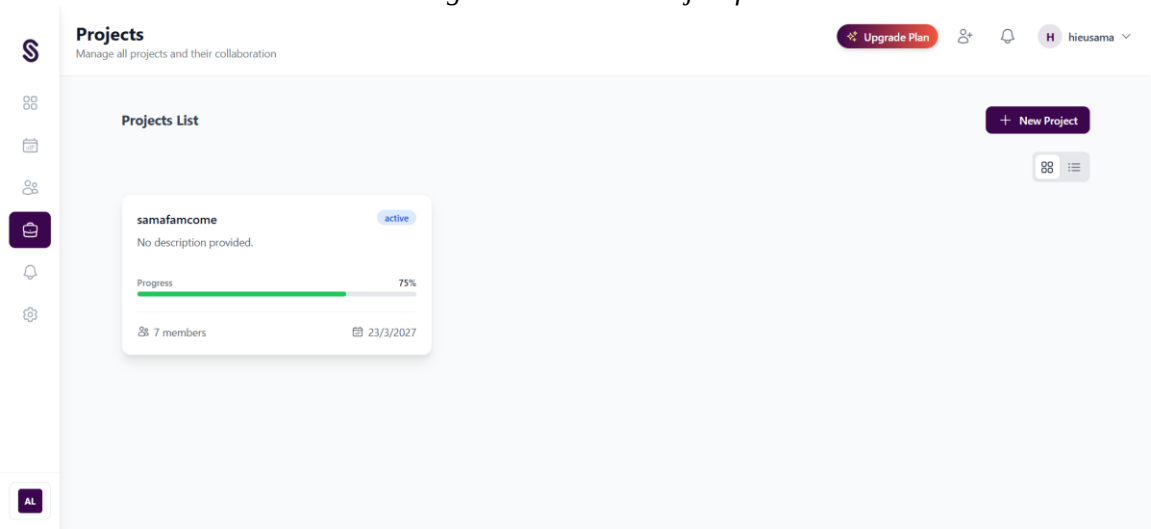


Figure 39: Project layout for Admin

6.3.2. Pages in MANAGER and Member role

6.3.2.1. Dashboard

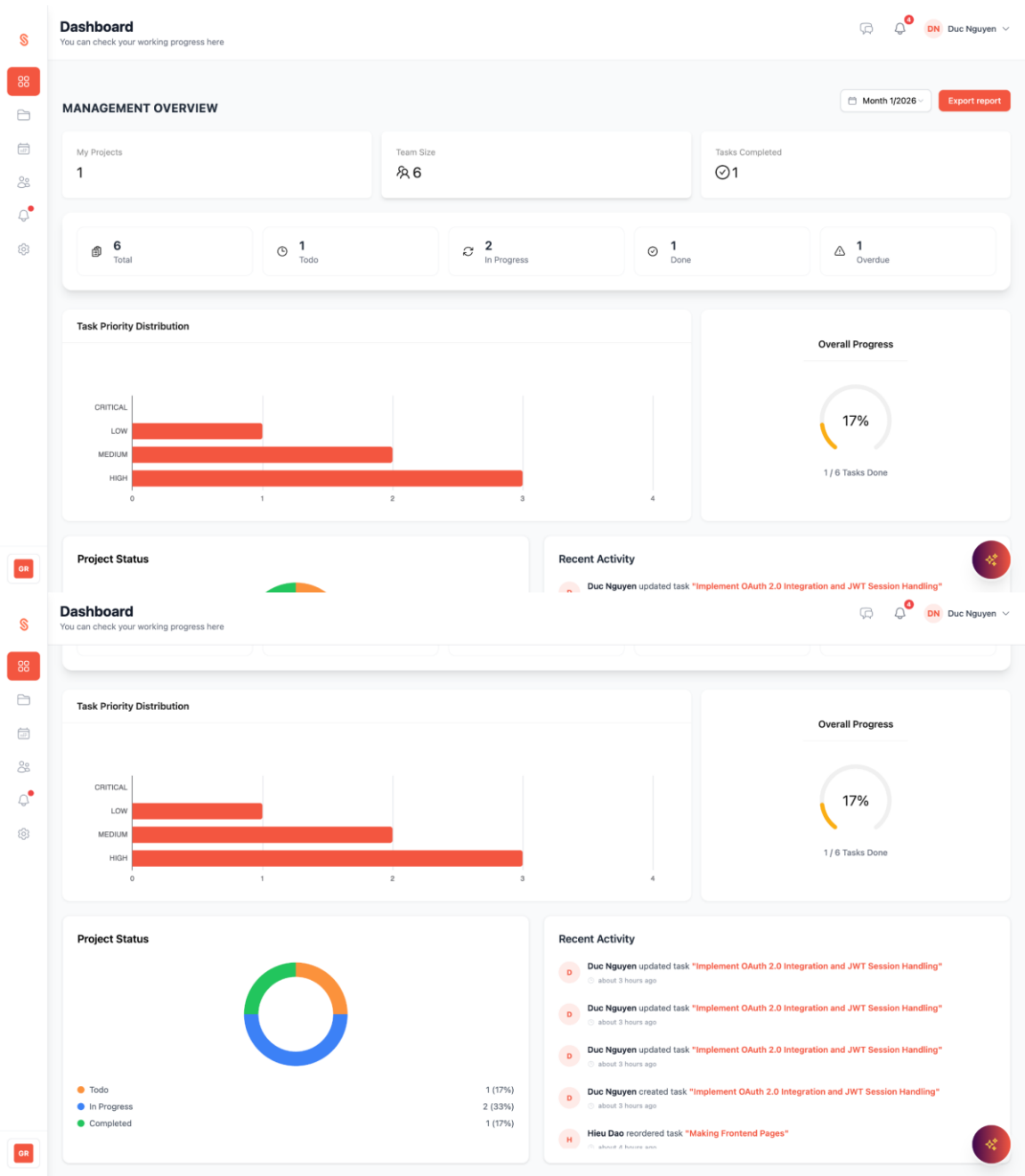


Figure 40 & 41: Dashboard layout for Manager

*Dashboard layout for Members looks the same but does not have My Projects, Team Size, Tasks Completed card.

6.3.2.2. Kanban Board

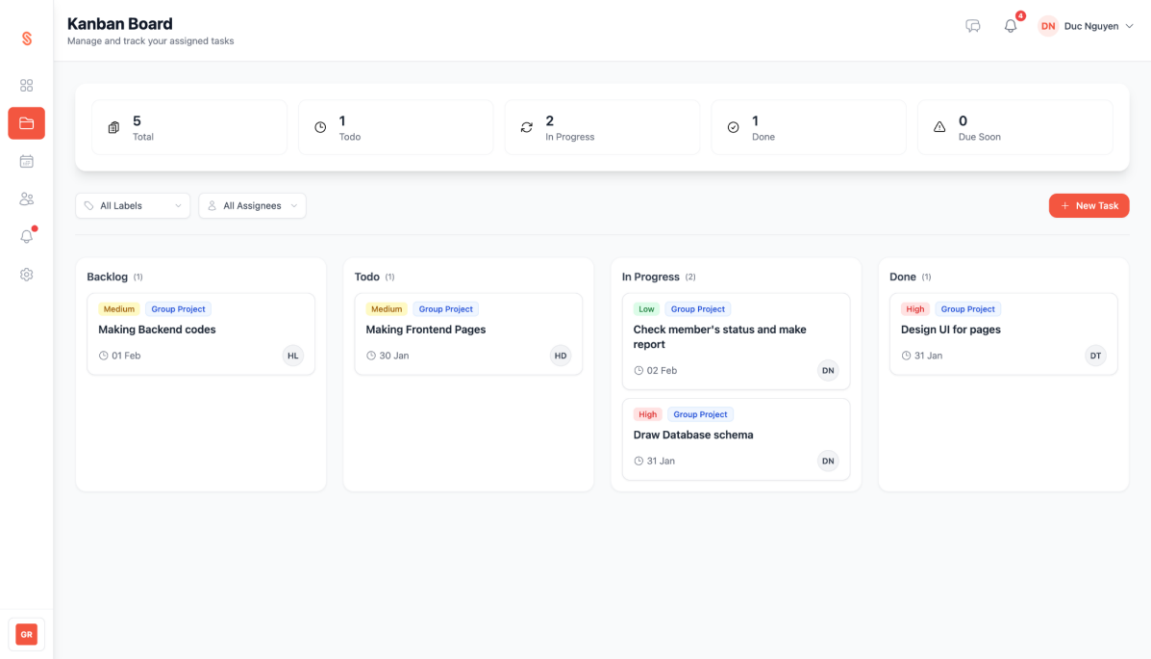


Figure 42: Kanban Board layout for Manager

*Kanban Board layout for Members looks the same but does not have a New Task button.

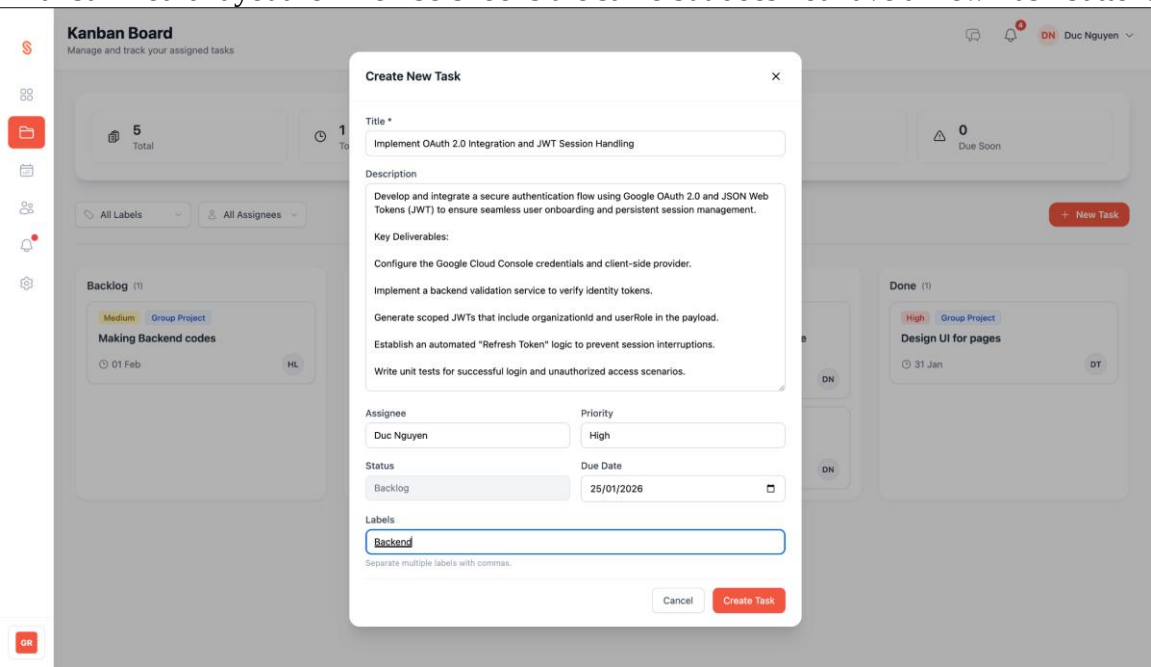


Figure 43: Form to make new task

6.3.2.3. Task Detail

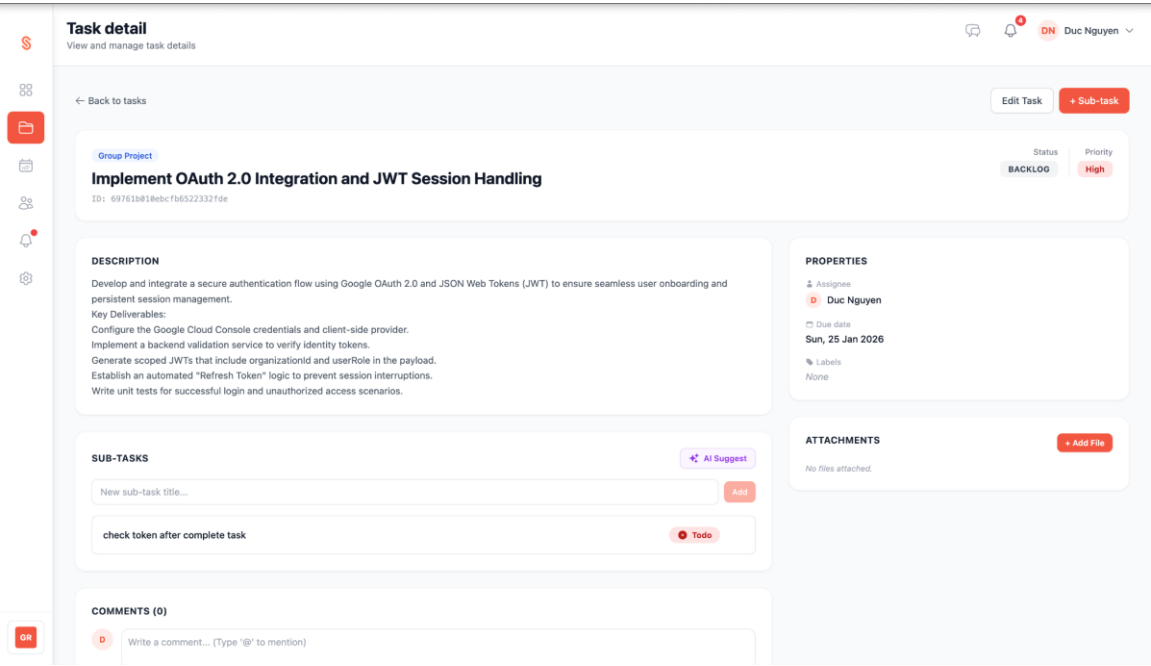


Figure 44: Task detail layout for Manager and Member

*Task detail layout for Members does not have an Edit Task button.

6.3.2.4. Calendar

*Calendar layout for Manager and Member looks like calendar layout for Admin but does not have Config IPs button.

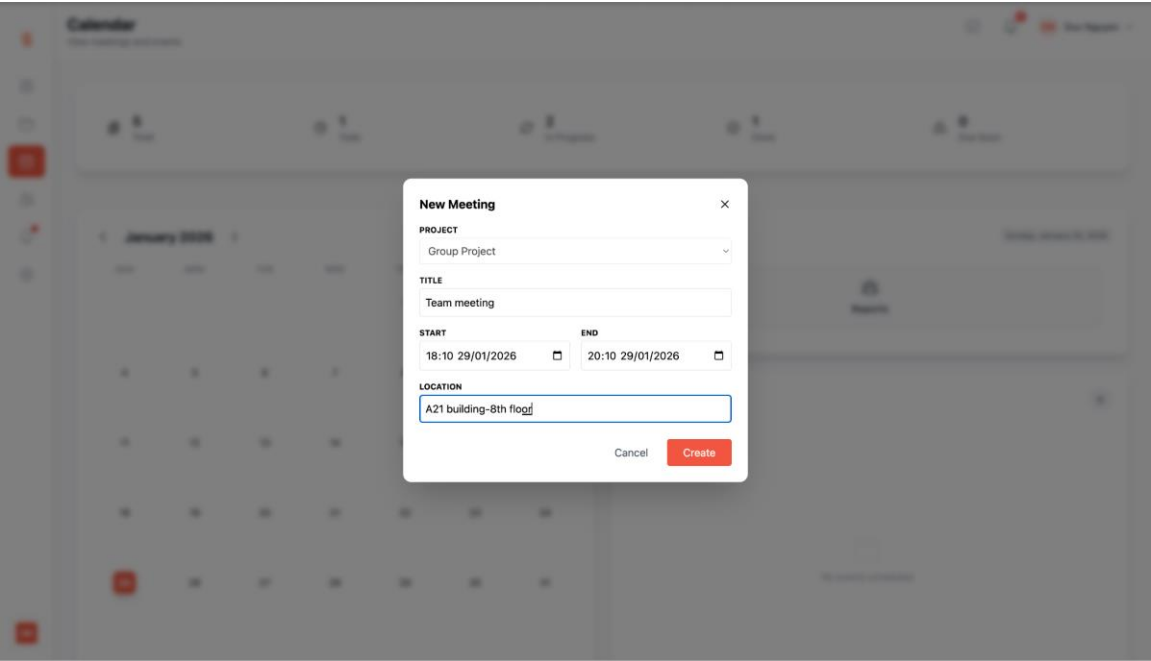


Figure 45: New Meeting form

6.3.2.5. Members

*Members layout for Manager and Member looks the same as Admin

6.3.3. Other pages

6.3.3.1. Notifications

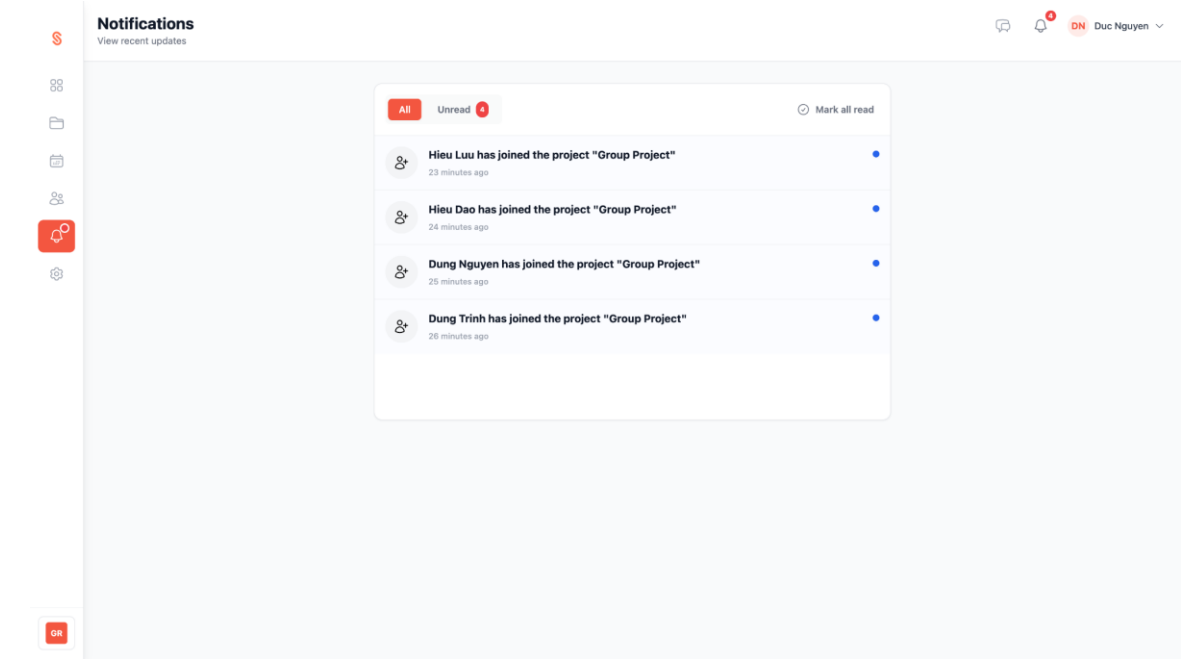


Figure 46: Notification layout for all User

6.3.3.2. Settings

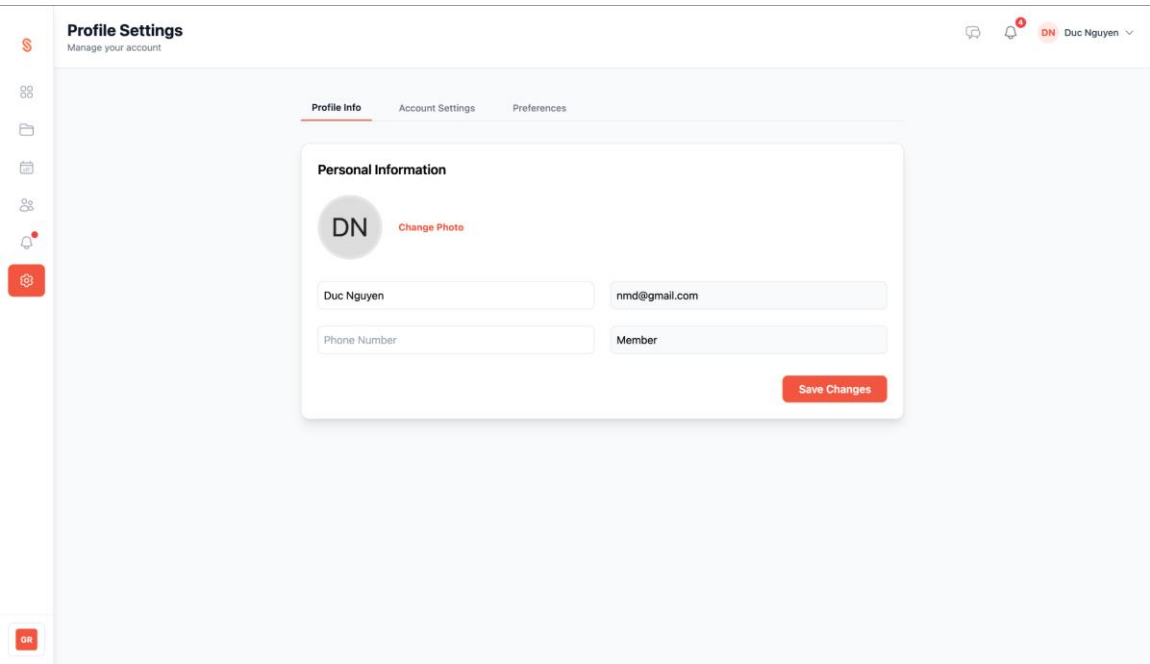


Figure 47: Personal Information

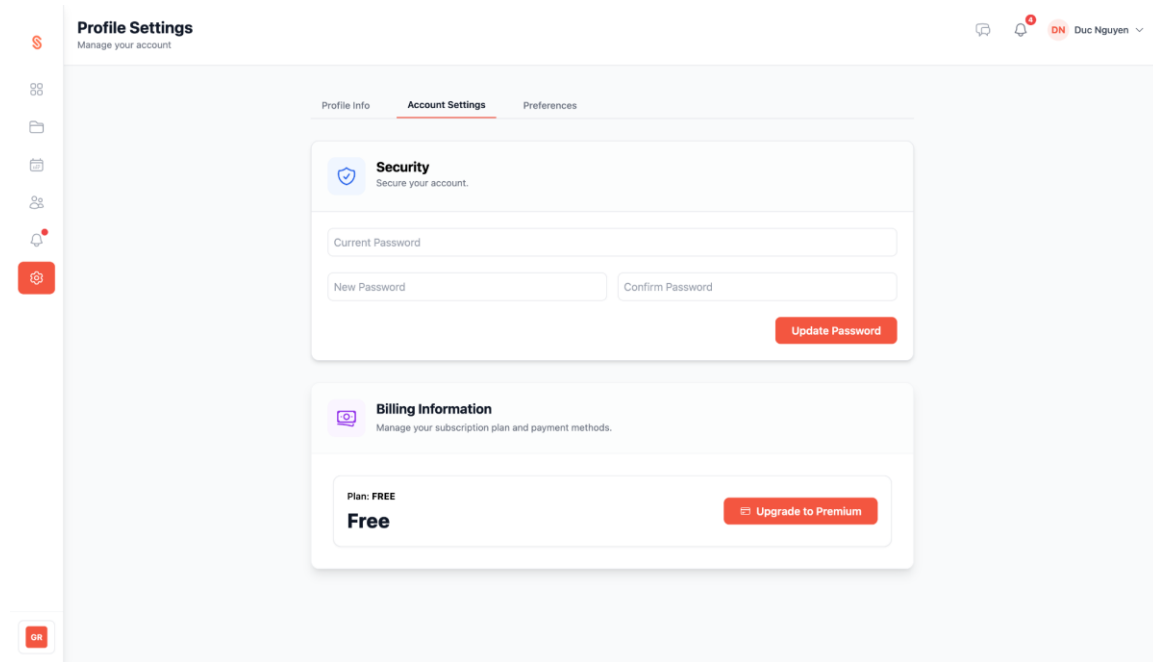


Figure 48: Account Settings

6.4. Discussion

The development and testing of the system have provided valuable insights into the efficiency of integrating AI and real-time communication within a project management framework. This section evaluates the current state of the application, focusing on key successes and inherent constraints.

6.4.1. Key Project Achievements

The system has successfully established a robust and scalable architecture, meeting the primary objectives defined in the functional requirements.

- **Robust Core Management:** The system successfully implements a Kanban workflow with smart indexing, allowing for fluid task reordering and real-time status updates.
- **Secure Multi-Tenant Architecture:** Through the implementation of the current Organization context and 3-Level RBAC, the system ensures strict data isolation between organizations, preventing unauthorized access across the workspace.
- **Seamless Real-time Collaboration:** The integration of Socket.io enables instantaneous communication through project-based chat rooms and immediate security responses, such as force-logout upon member removal.
- **AI-Enhanced Productivity:** The Magic Subtask and Daily Smart Summary features successfully demonstrate the utility of AI in breaking down complex tasks and providing personalized briefings, thereby reducing the cognitive load on team members.

6.4.2. Limitations

Despite the successful implementation of core features, several limitations were identified during the testing and evaluation phases that represent areas for future enhancement.

- **Plan Quota Restrictions:** While the Free plan serves as a gateway for new users, the strict limit of one project can be restrictive for small teams wanting to test the system across multiple departments.
- **Environmental IP Dependency:** The attendance system is heavily reliant on a static IP whitelist. This poses a challenge for hybrid or remote-first organizations where members may not be physically present at a single authorized office location.
- **External Service Dependency:** The stability of the Payment (Stripe) and AI (Magic Subtask) modules is contingent on the availability and API response times of third-party providers. Any external downtime directly impacts these critical system functions.
- **UI/UX Scalability:** While the Kanban board is effective for standard project loads, managing a high volume of tasks (over 100 per column) may lead to performance degradation or a cluttered interface without advanced filtering options.

7. CONCLUSION AND FUTURE WORK

7.1. Conclusion

In conclusion, this project successfully developed and deployed a comprehensive SaaS-based Project Management and Collaboration System specifically tailored for Small and Medium Enterprises (SMEs). By utilizing the MERN stack (MongoDB, Express.js, React.js, Node.js), the system effectively addressed the critical challenges of remote work management, namely the fragmentation of tools and the lack of visibility into employee productivity.

Summary of Key Achievements:

- **Robust Multi-tenant Architecture:** The system successfully implemented a secure B2B multi-tenant architecture. Data isolation was achieved at the database level using organizationID, ensuring that company workspaces remain private and secure while sharing the same cloud infrastructure.
- **Integrated Digital Office Concept:** Unlike traditional task trackers, this project bridged project management with human resources by integrating an IP-based Attendance System and a Meeting Scheduler with Conflict Detection. This provides SMEs with a unified platform for both operation execution and workforce monitoring.
- **Advanced Role-Based Access Control (RBAC):** The system successfully implemented a dual-layered permission model. At the Frontend level, dynamic dashboards automatically adapt to display relevant KPIs and charts based on whether the user is an Organization Admin, Project Manager, or Member. At the Backend level, strict authorization logic prevents unauthorized data manipulation.
- **Practical AI Integration:** The project successfully integrated Generative AI to enhance user productivity. Features such as "Magic Subtasks" (automated task breakdown) and the "AI Daily Brief"(intelligent schedule summarization) were implemented with rate-limiting protection.
- **System Reliability:** Critical workflows, such as user onboarding and project joining via invite codes, were fortified using MongoDB Transactions (ACID compliance). This prevents data inconsistency during concurrent operations.

Overall, the project met all its initial objectives, delivering a functional, secure, and modern workspace solution that is ready for real-world application.

7.2. Future Work

To ensure the system's long-term scalability and to provide a flexible, reliable user experience, the next phase of development will focus on two key areas: strategic architectural migration and multi-platform expansion.

7.2.1. Microservices Migration

The current monolithic architecture, while effective for rapid initial deployment, poses a challenge for extreme scaling. A strategic migration to a Microservices architecture is planned to:

- **Improve Scalability & Resilience:** Decouple core functions (Task Management, Real-time Communication, AI Services) into independent services. This allows for

horizontal scaling of individual components based on load and localizes failure to prevent system-wide outages.

- **Enhance Technology Flexibility:** Enable the use of the most optimal technology stack for each service, moving beyond the current MERN stack constraints for specialized tasks.
- **Advanced Attendance Verification:** To enhance the accuracy and security of the check-in process.

7.2.2. Dedicated Mobile Application

To support remote and on-the-go productivity, the project will develop dedicated native mobile applications (iOS and Android). This will facilitate:

- **Optimized User Experience:** Provide a seamless interface for critical daily activities like viewing the Dashboard and updating Task Status.
- **Reliable Real-time Communication:** Integrate native push notifications for immediate, dependable alerts on task assignments and mentions.
- **Flexible Attendance:** Extend the secure IP Check-in feature to include mobile-centric authentication methods, such as GPS location tracking, fully supporting hybrid and remote work models.

References

- [1] Ahmed Patel, “Design of an Integrated Project Management Information System for Large Scale Public Projects”, 2011.
- [2] Wojciech Zabierowski, “Web-based Project Management System”, 2009.
- [3] Anshilin iyyadurai Nadar, “Project Management System”, 2024
- [4] Sine Meyer Jakobsen a, Mathias Brinkmann Momme, “Exploring strategies for successful implementation of IT projects: integrating project management practices and human factors”,2025
- [5] Verma, V. K. (1996). Human resource management and project management. PM Network, 10(4), 34–40.
- [6] Nurwidyantoro, A., Shahin, M., Chaudron, M., Hussain, W., Perera, H., Shams, R.A., Whittle, J.: Towards a Human Values Dashboard for Software Development: An Exploratory Study. Association for Computing Machinery, New York, NY, USA (2021)
- [7] Robert Ferguson, “How to Develop And Use a Project Dashboard” ,2018